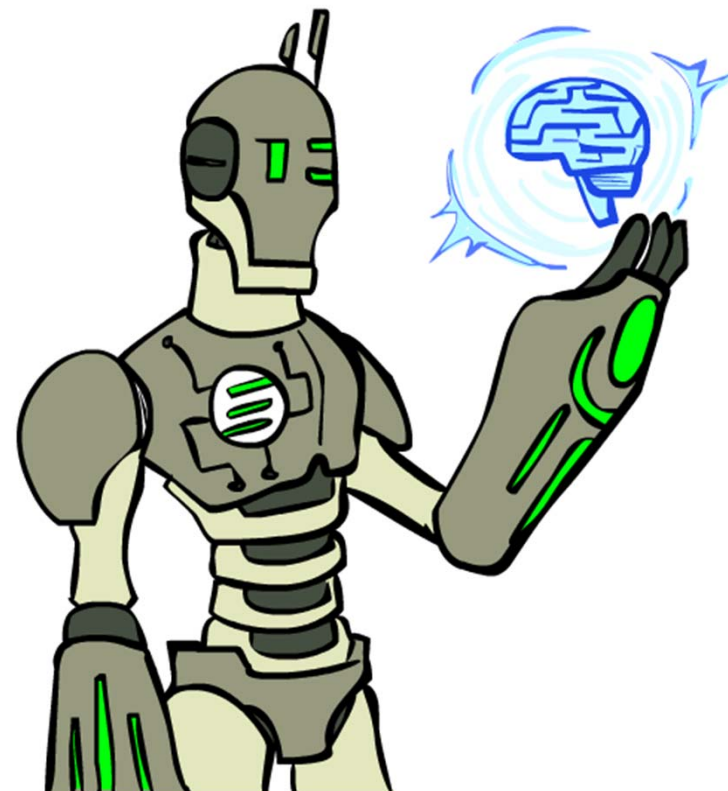


第 2 部分 搜索问题求解

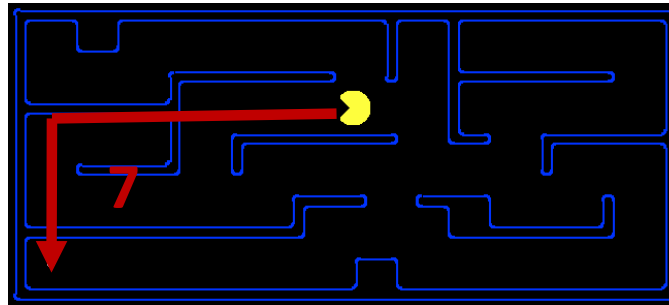
第3讲 搜索问题求解

- 2.1 问题求解智能体
- 2.2 问题示例
- 2.3 搜索算法
- 2.4 无信息搜索策略
- 2.5 有信息（启发式）搜索策略
- 2.6 启发式函数



设计启发函数

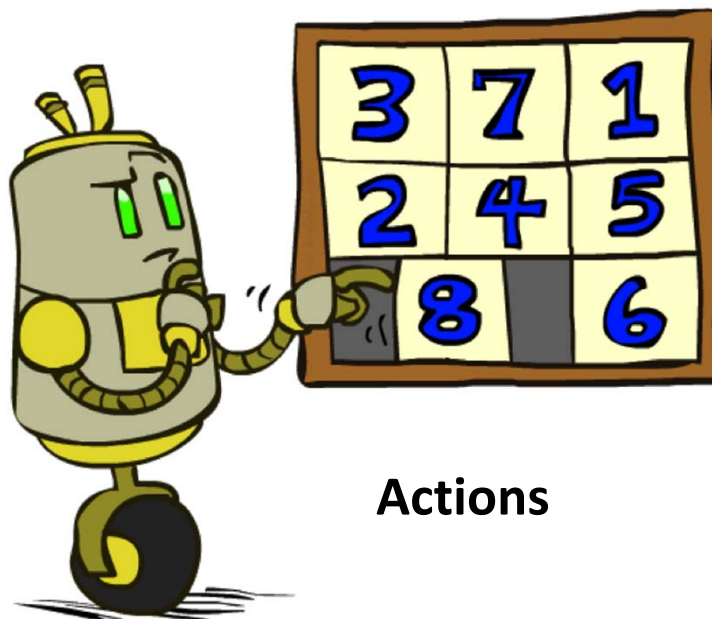
- 启发函数的准确性是如何影响搜索性能的？
- 如何构造启发式函数？
- 以8数码问题为例



例子: 8数码问题

7	2	4
5		6
8	3	1

Start State



Actions

	1	2
3	4	5
6	7	8

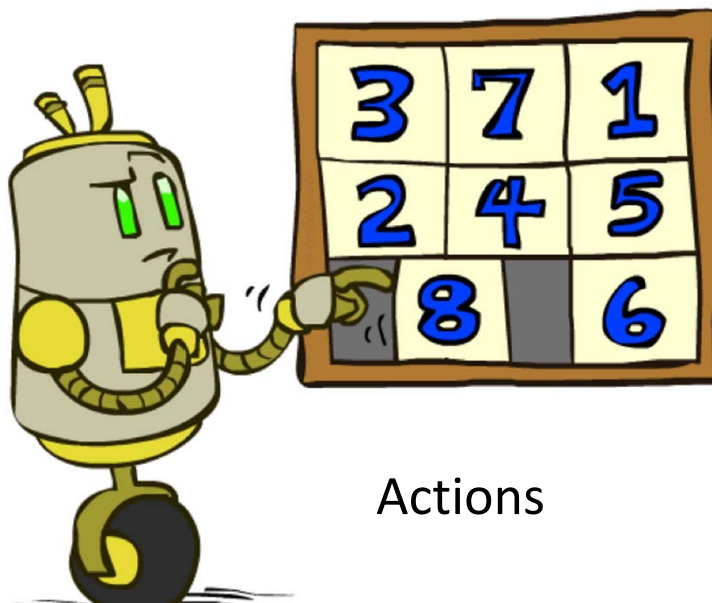
Goal State

- ☐ What are the states(状态)?
- ☐ How many states? (状态数) $9!/2$ 进一步补充
- ☐ What are the actions? 空位左右上下移动
- ☐ How many successors/后继 from the start state?
- ☐ What should the costs be? 每一步为1, 整个路径的值为路径中的步数

例子: 8数码问题

7	2	4
5		6
8	3	1

Start State



Actions

	1	2
3	4	5
6	7	8

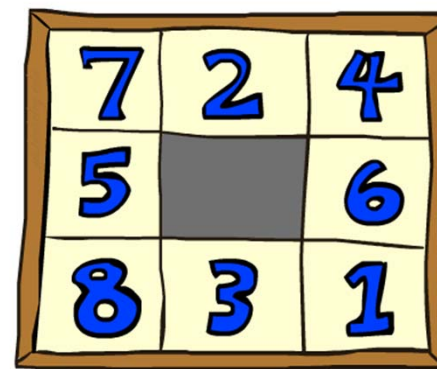
Goal State

如果方格X与方格Y相邻，且Y是空格，那么滑块可以从方格X移动到方格Y。

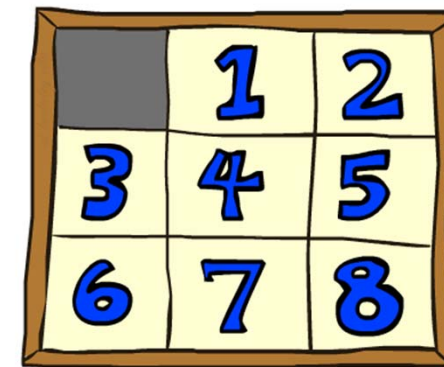
- **松弛问题【减少对动作的限制条件的问题】**
- (a) 如果方格X与方格Y相邻，那么滑块可以从方格X移动到方格Y。【曼哈顿距离】
- (b) 如果方格Y是空格，那么滑块可以从方格X移动到方格Y。
- (c) 滑块可以从方格X移动到方格Y。【错位滑块】

8数码问题 I

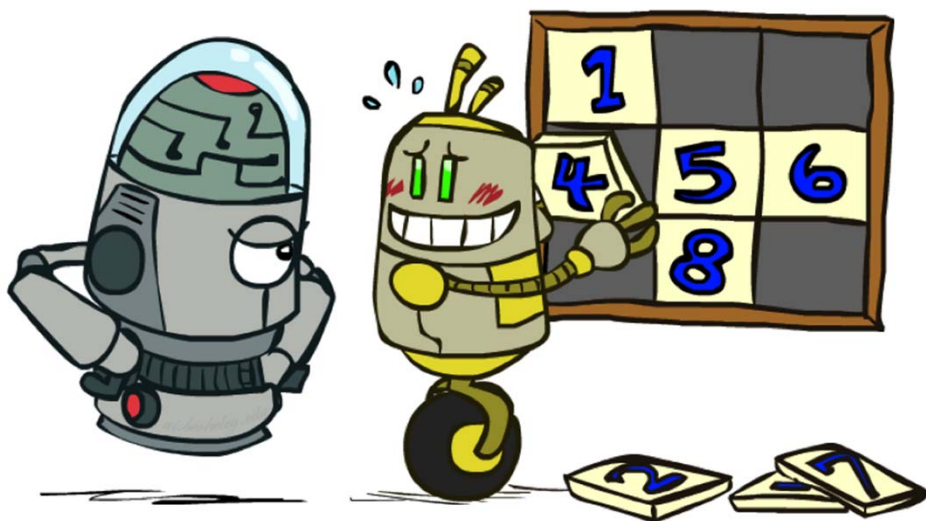
- $h1$ = 错位滑块的数量 (不包括空格)
- $h1(\text{start}) = ?$ 8
- 为什么是可容的?



Start State



Goal State



Average nodes expanded when the optimal path has...			
	...4 steps	...8 steps	...12 steps
UCS	112	6,300	2.6×10^6
TILES	13	39	227

Statistics from Andrew Moore

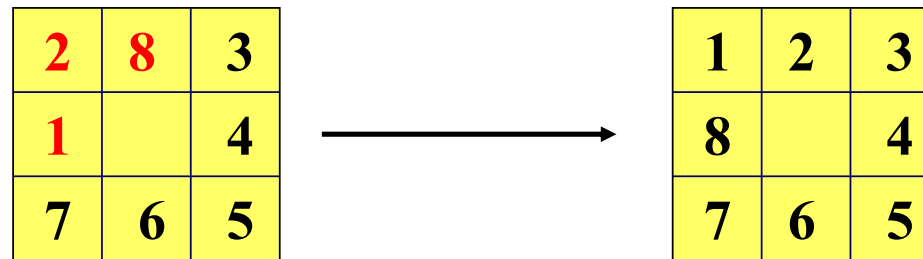
■ 例1： 8数码问题 I

(1) 估值函数 $f(n)$ 设置：

$$f(n) = d(n) + h1(n)$$

$d(n)$ ： 节点 n 的深度 $h1(n)$ ： 错位滑块数

(2) 如下的八数码难题(8-puzzle problem)



(初始状态)

(目标状态)

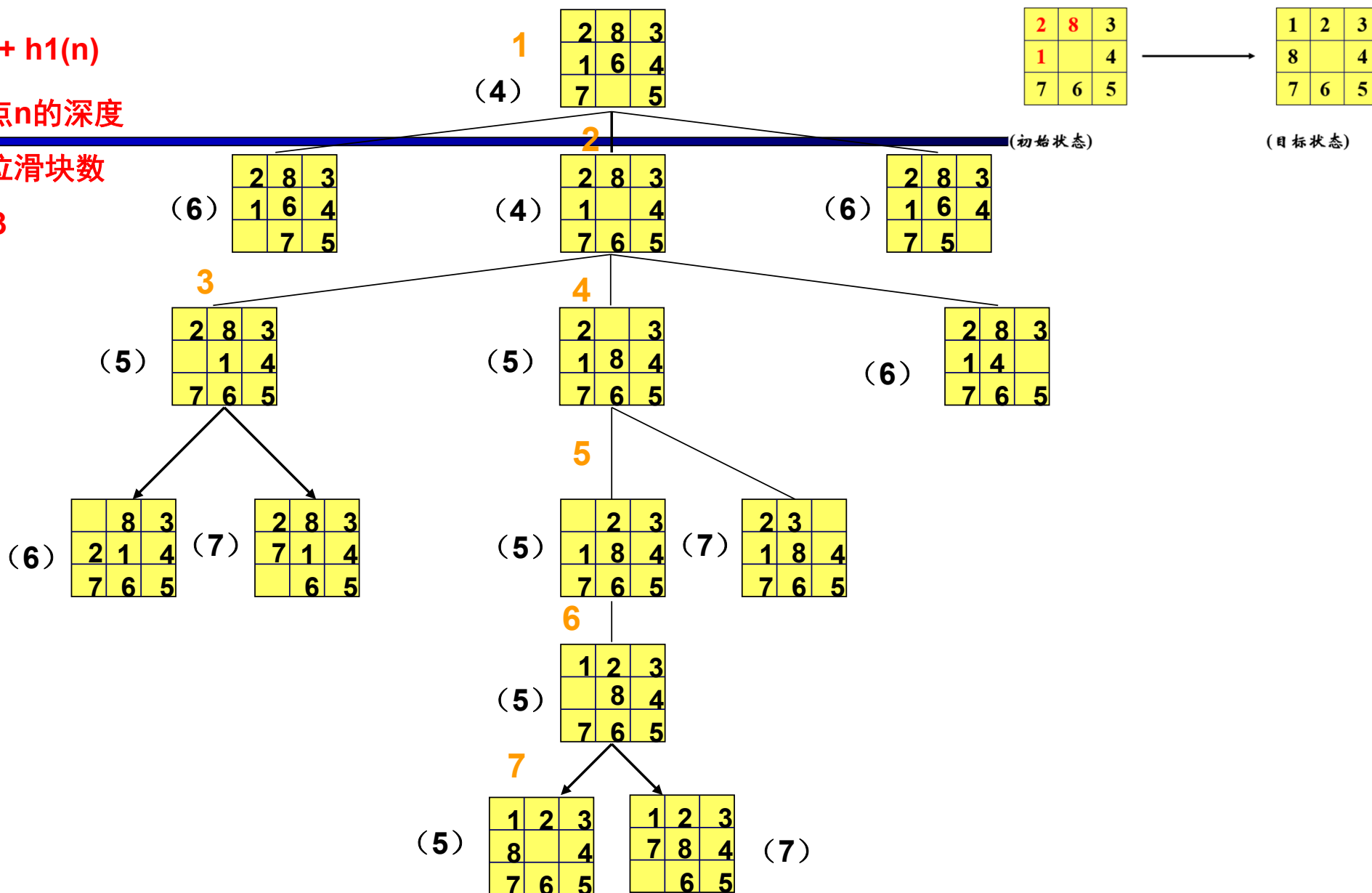
(3) 八数码难题 $h(n) = h1(n)$ 的搜索树见下图：

$$f(n) = d(n) + h1(n)$$

$d(n)$: 节点n的深度

$h1(n)$: 错位滑块数

$h1(\text{Start})=3$



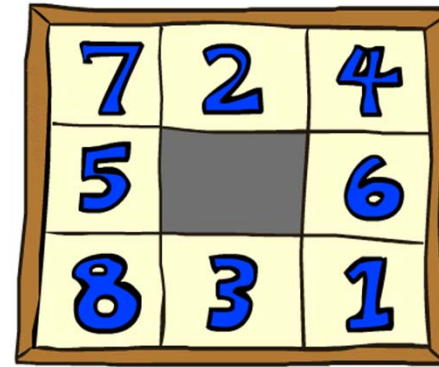
8数码问题 II

□ h_2 = 滑块到其目标位置距离的总和
(曼哈顿距离)

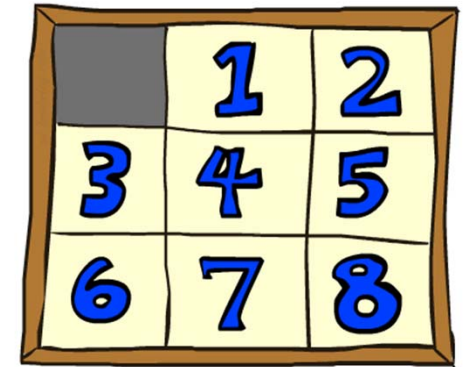
□ 为什么是可容的?

□ $h_2(\text{start}) = 3 + 1 + 2 + 2 + 2 + 2 + 3 + 2 = 18$

7: 开始位置 (0, 0),
目标位置 (2, 1),
所以 $|2-0| + |1-0| = 3$



Start State



Goal State

Average nodes expanded when
the optimal path has...

	...4 steps	...8 steps	...12 steps
TILES	13	39	227
MANHATTAN	12	25	73

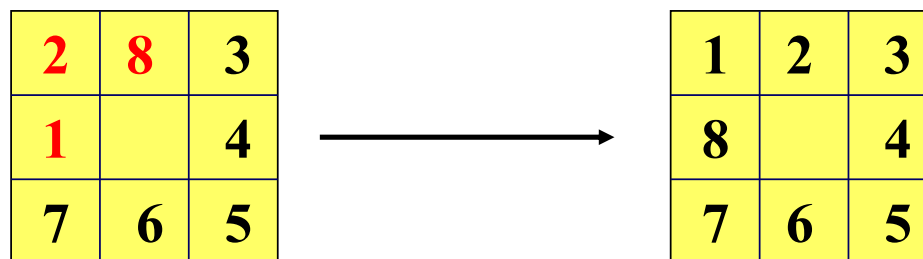
■ 例2： 8数码问题II

(1) 估值函数 $f(n)$ 设置：

$$f(n) = d(n) + h2(n)$$

$d(n)$ ： 节点 n 的深度 $h2(n)$ ： 曼哈顿距离

(2) 如下的八数码难题(8-puzzle problem)



(初始状态)

(目标状态)

(3) 八数码难题 $h(n) = h2(n)$ 的搜索树见下图：

$$f(n)=d(n)+h2(n)$$

$d(n)$: 节点深度

$h2(n)$: 曼哈顿距离

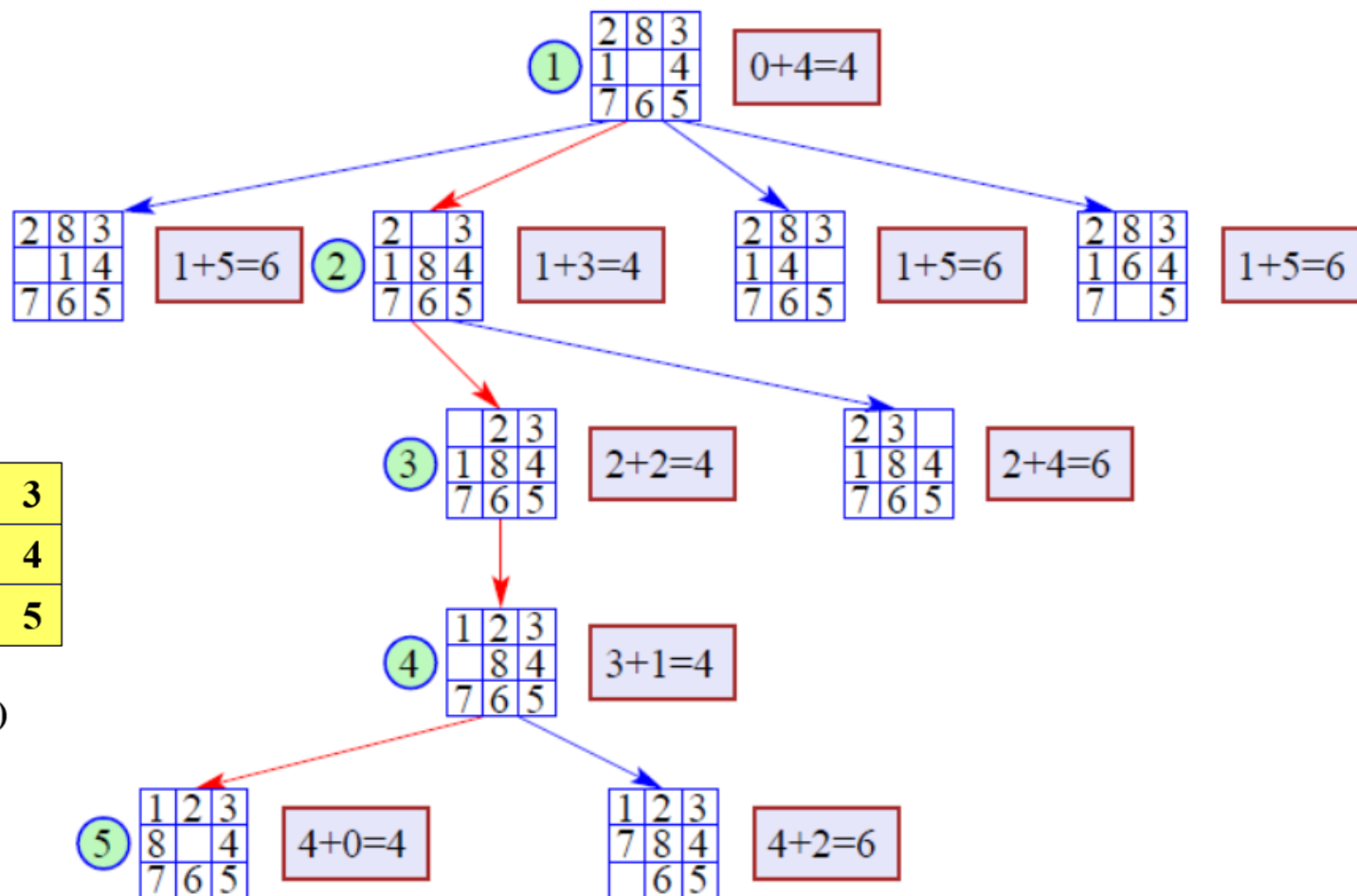
$$h2(\text{Start})=4$$

2	8	3
1		4
7	6	5

(初始状态)

1	2	3
8		4
7	6	5

(目标状态)



$h2 > h1$, $h2$ 比 $h1$ 少2个搜索节点,
 $h2(n)$ 比 $h1(n)$ 带有更多启发信息。

A*算法的最优性

A*算法的搜索效率很大程度上取决于启发函数 $h(n)$ 。

一般来说，在满足 $h(n) \leq h^*(n)$ 的前提下，

$h(n)$ 的值越大，

说明它携带的启发性信息越多，

A*算法搜索时扩展的节点就越少，

搜索效率就越高。

A*算法的这一特性被称为最优性。

8数码问题III

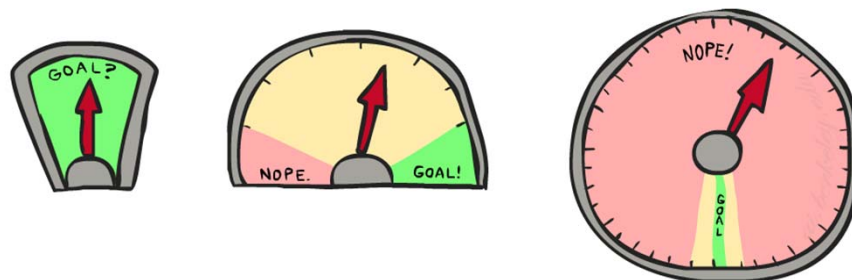
□ 如果使用实际代价作为启发式方法怎么样？

□ 它是否满足可容性？ $h(n) \leq h^*(n)$

□ 是否能减少扩展的节点数量？

□ 会节省扩展节点？

□ 这其中有什么问题？

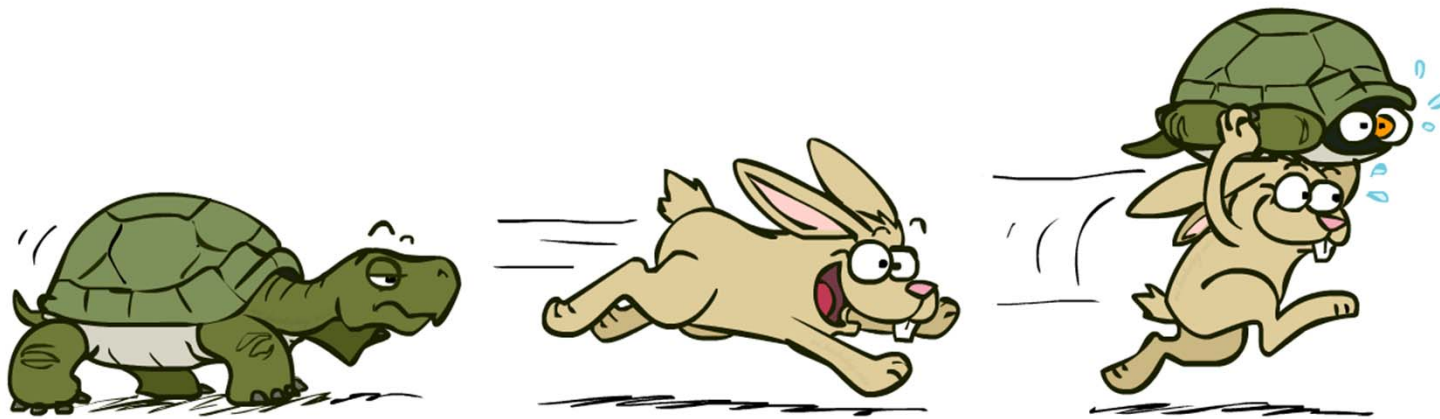


□ 使用A*算法：估算的质量与每个节点的工作量之间存在权衡。

□ 当启发式方法越接近真实代价时，你将扩展更少的节点，但通常需要为每个节点计算启发式本身而做更多的工作【找到这个 $h^*(n)$ 不容易】。

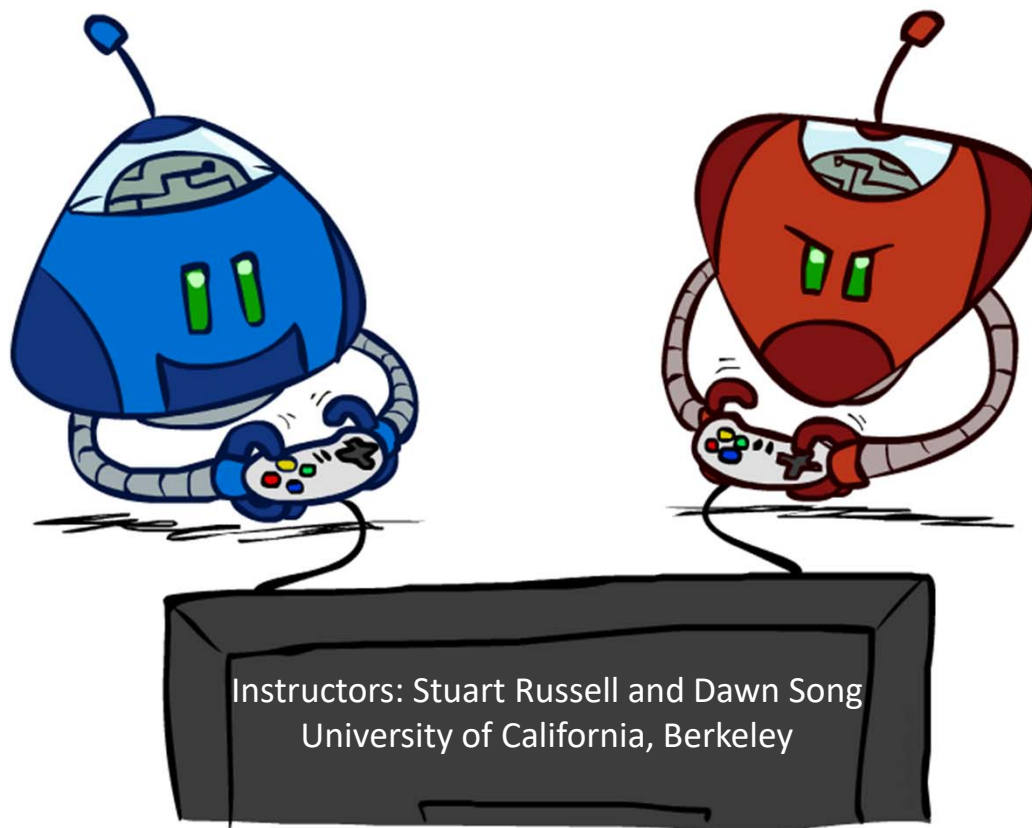
A*算法: 总结

- A*算法使用 $f(n)=g(n)+h(n)$ 的代价函数形式。
- 在可容的启发式下，A*是最优的。
- 启发式设计是关键：经常使用松弛方式来设计。



-
- **上述搜索算法：**
 - **自己一个人在那默默无闻的东奔西走、上下求索！**

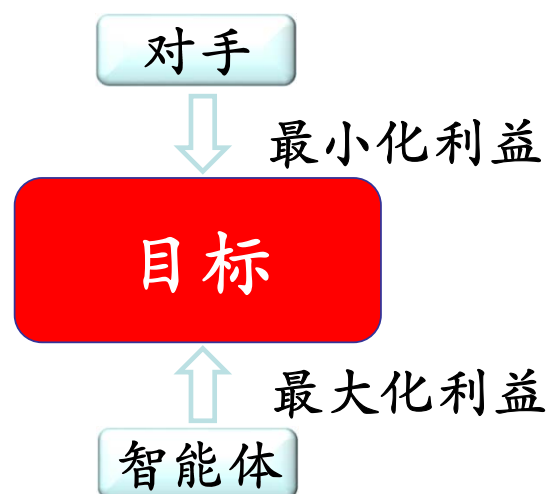
对抗搜索和博弈



对抗搜索

- 对抗搜索 (Adversarial Search) 也称为博弈搜索 (Game Search)
- 在一个竞争环境 (Competitive Environment) 中，两个或两个以上的智能体 (agents) 之间通过竞争实现相反的利益，一方最大化这个利益，另外一方最小化这个利益。

狭路相逢勇者胜
勇者相逢智者胜
智者相逢德者胜
德者相逢道者胜

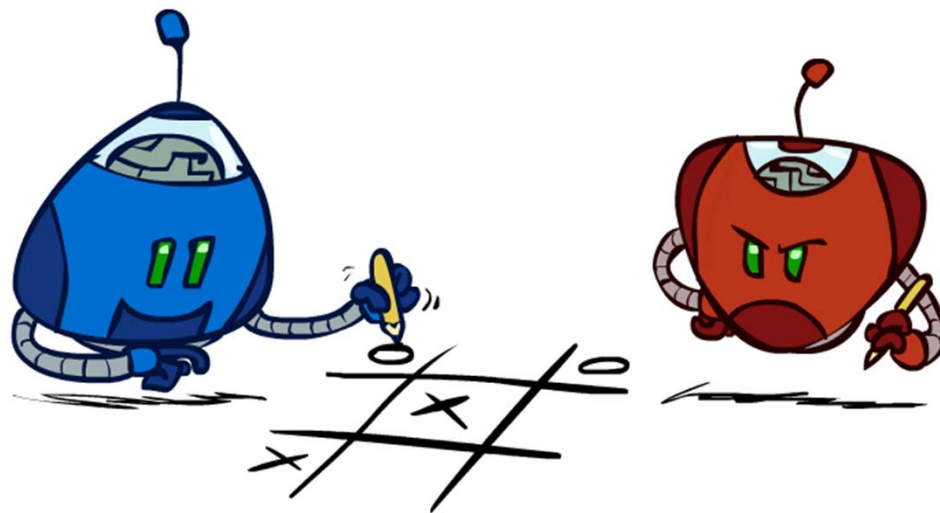


博弈论

- 用**对抗博弈树搜索技术**显式地对对抗智能体建模。
- 定义最优移动并寻找最优移动的算法——**极小化极大搜索**（minimax Search）。
- **剪枝**（pruning）通过忽略搜索树中对最优移动没有影响的部分来提高搜索效率。

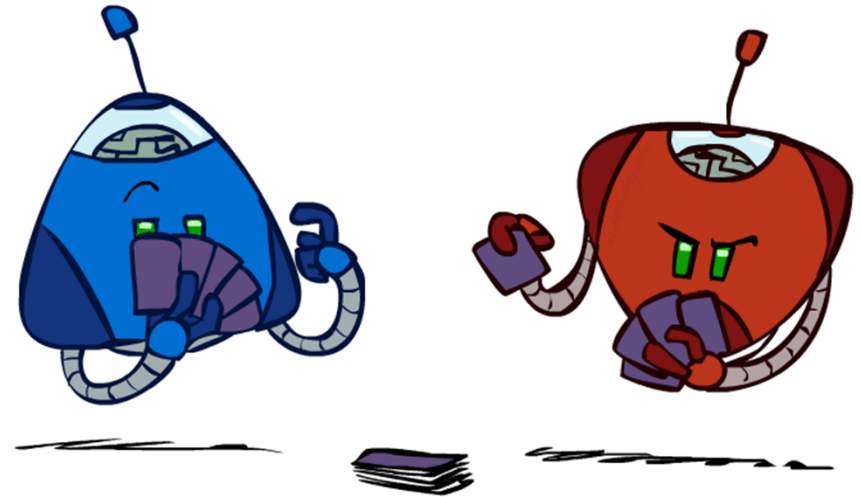
概要

- 极小化极大搜索算法
- α - β 剪枝算法

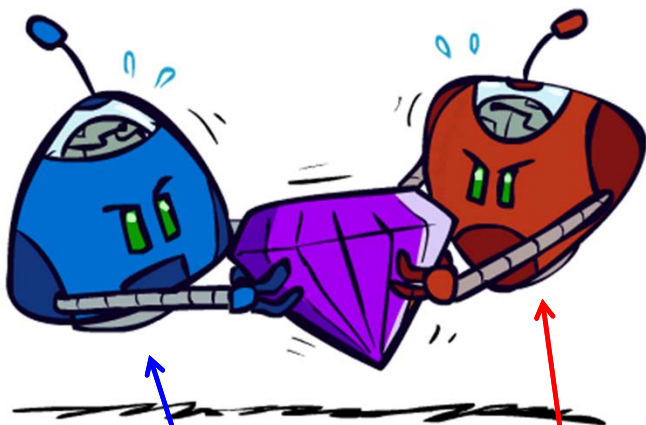


游戏的类型

- 游戏 = 任务环境中有两个或多个智能体
 - 确定性的还是随机性的？
 - 完美信息（完全可观察）？
 - 一、二或更多玩家？
 - 轮流还是同时？
 - 零和？（对一方有利的东西将对另一方同等程度有害：不存在“双赢”结果）
- 希望设计出一种可应变计划的算法（也称为策略或政策），它为每种可能的情况推荐一个动作。



博弈



■ 零和博弈:

- 智能体具有**相反**的效用
- 纯粹的竞争:
 - 一方最大化, 另一方最小化。
One *maximizes*, the other *minimizes*
 - 如国际象棋、围棋等



■ 通用游戏

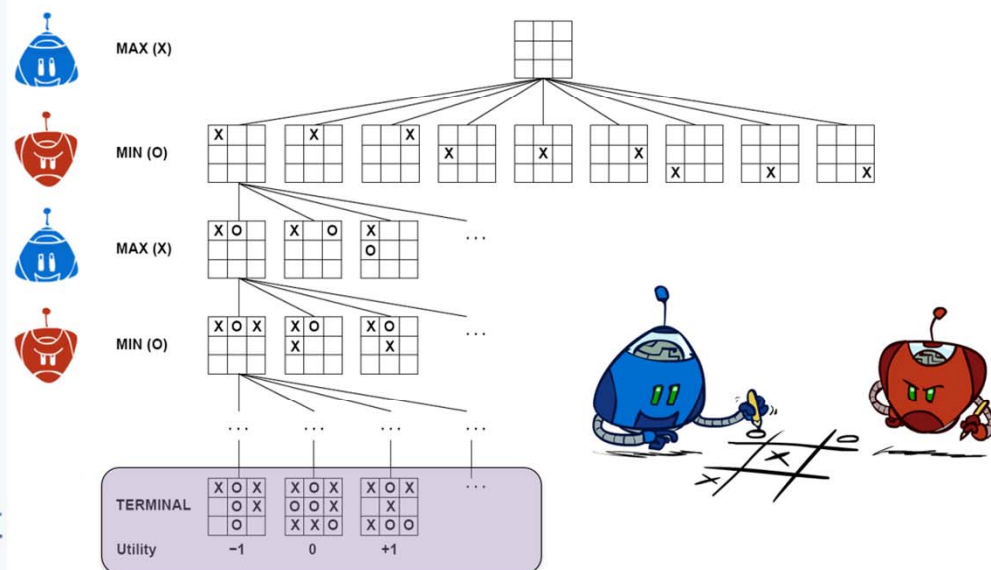
- 智能体具有独立的效用
 - 合作、冷漠、竞争、不断变化的联盟等都是可能的。
 - Cooperation, indifference, competition, shifting alliances, and more are all possible
 - 如投硬币

双人零和博弈

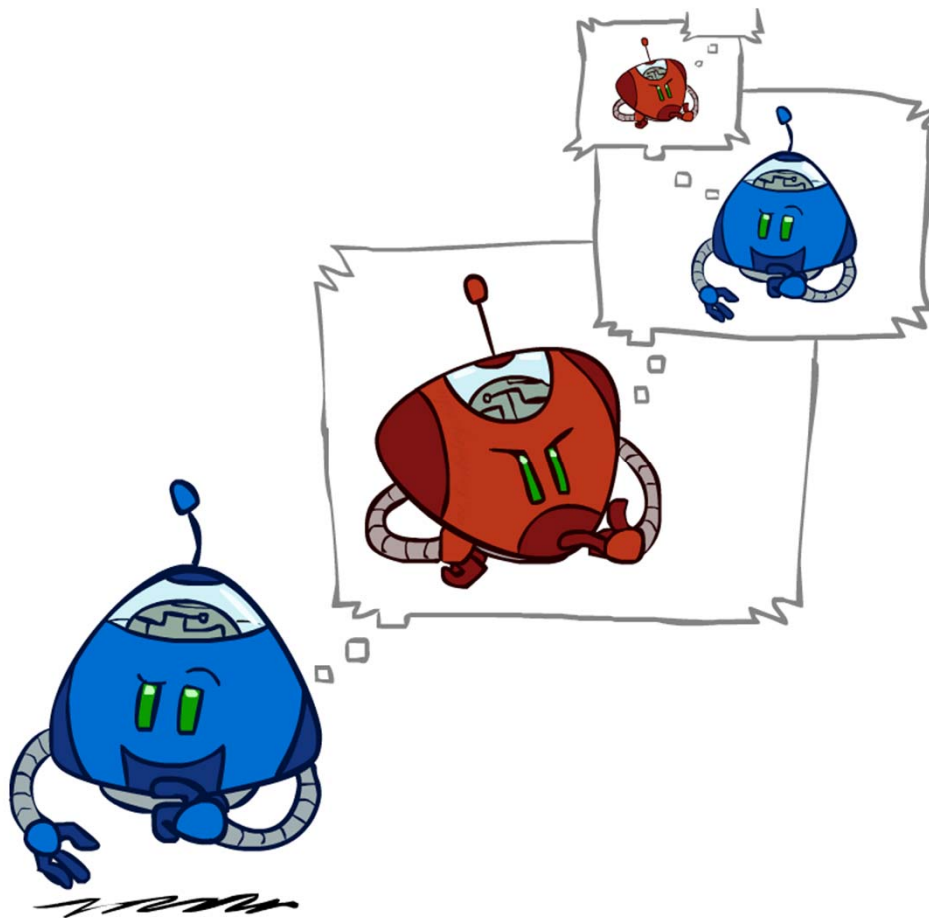
双人零和博弈(zero-sum game): 确定性、双人、轮流、完美信息(perfect information).
移动(move) 作为“动作”(action) 局面(position) 作为“状态”(state)

- S_0 : **初始状态**, 指定博弈开始时如何设置。
- $TO-MOVE(s)$: 在状态 s 下, 轮到其移动的参与者。
- $ACTIONS(s)$: 在状态 s 下, 全体合法移动的集合
- $RESULT(s, a)$: **转移模型**, 定义状态 s 下执行动作 a 所产生的结果状态。
- $IS-TERMINAL(s)$: **终止测试** (terminal test), 博弈结束时返回真, 否则返回假。博弈结束时的状态称为**终止状态** (terminal state)。
- $UTILITY(s, p)$: **效用函数** (也称为目标函数或收益函数), 定义博弈结束时终止状态 s 下参与者 p 得到的最终的数值收益。在国际象棋中, 结果为赢、输或平局, 收益分别为1、0或1/2。^[4]一些博弈存在更大范围

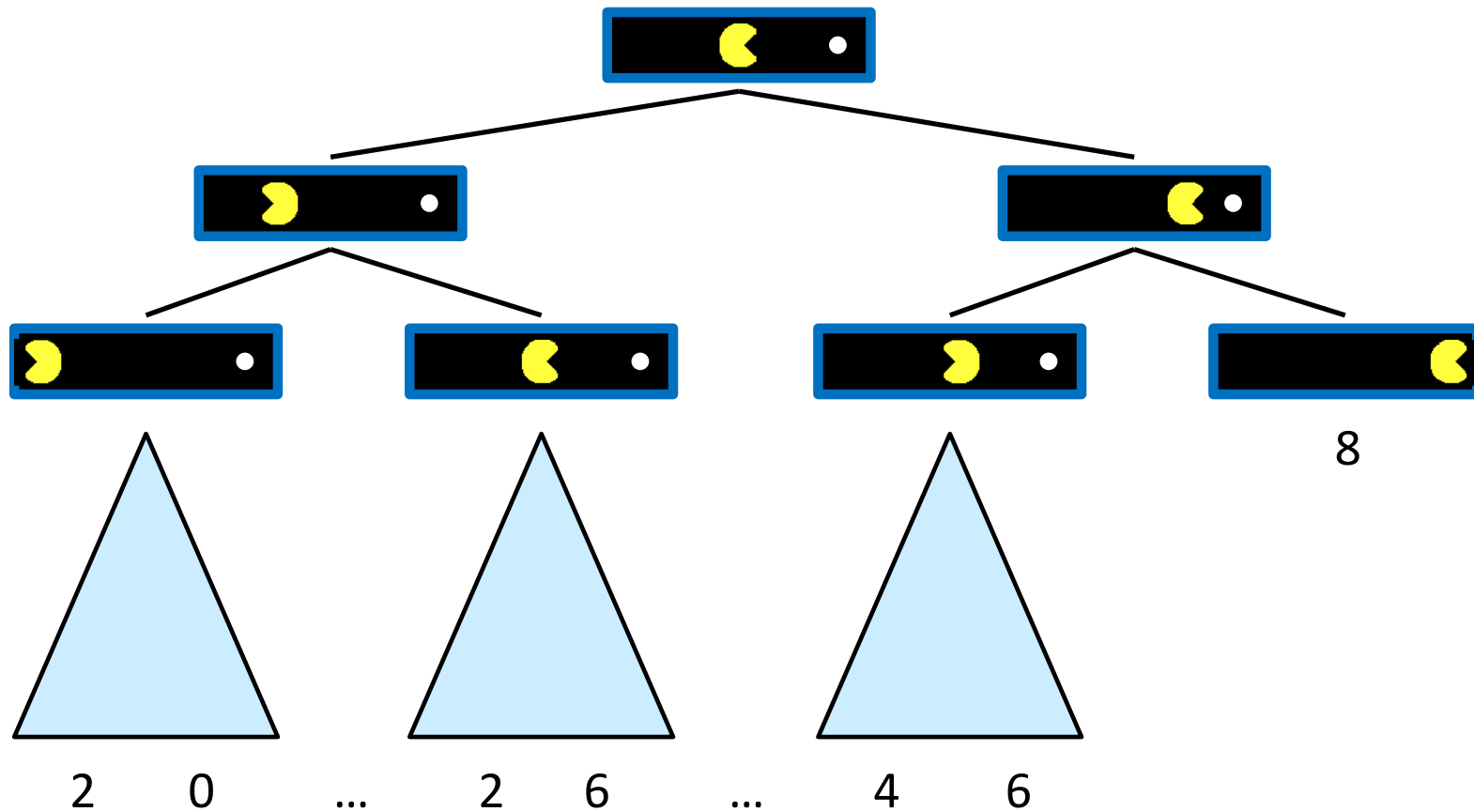
井字棋 (圈叉游戏tic-tac-toe) 部分博弈树



对抗搜索

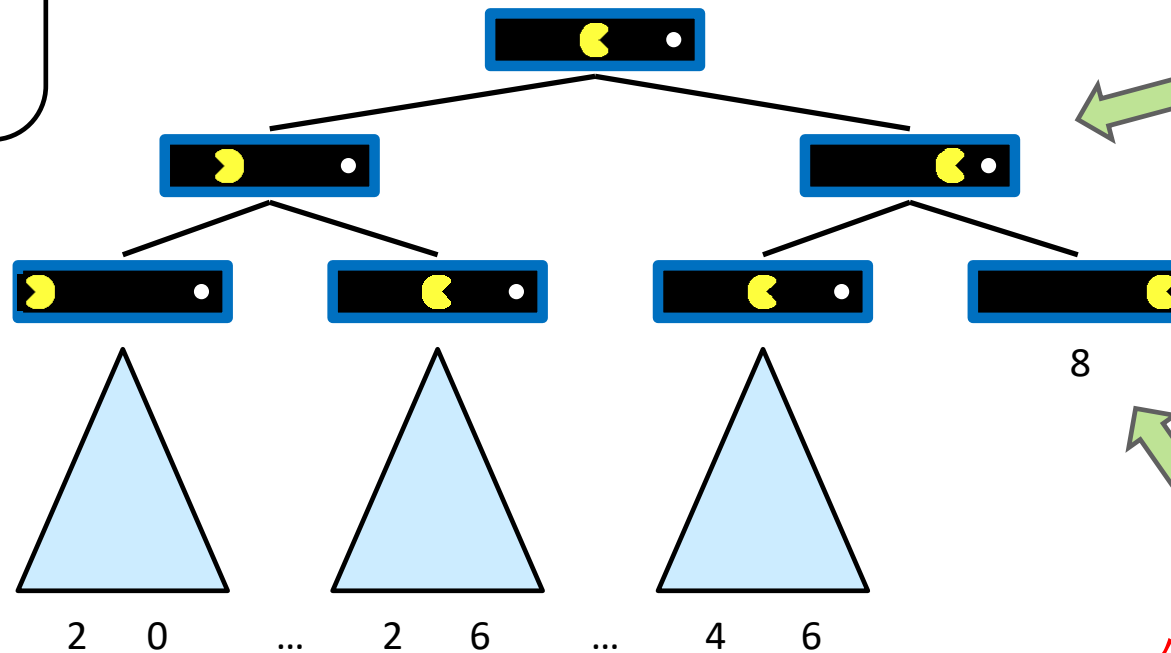


单智能体树



状态的值

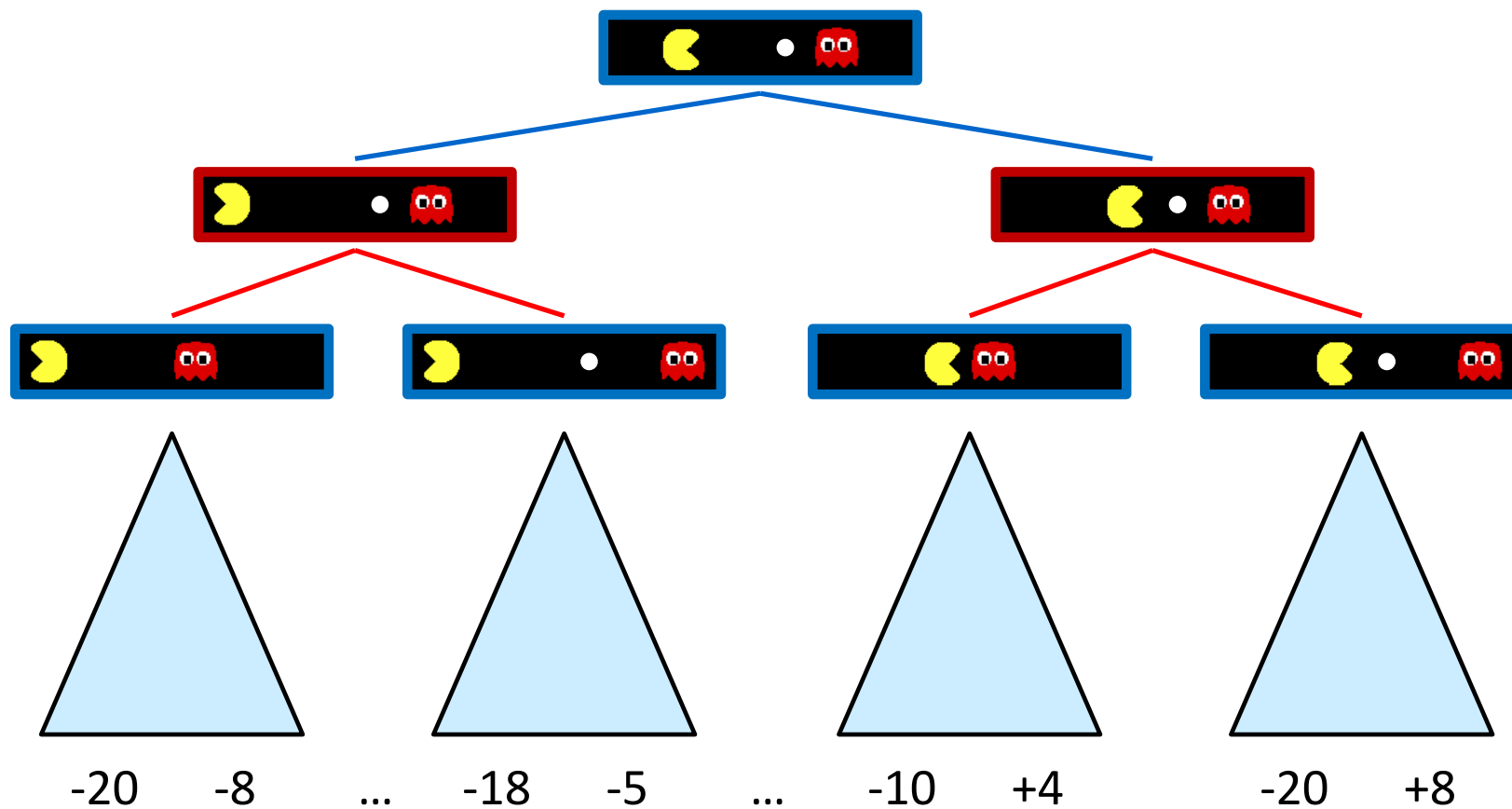
状态的值：
从该状态可达的
最佳结果（效用）



非终止状态：
 $V(s) = \max_{s' \in \text{successors}(s)} V(s')$

终止状态：
 $V(s) = \text{known}$

对抗博弈树



极小化极大值

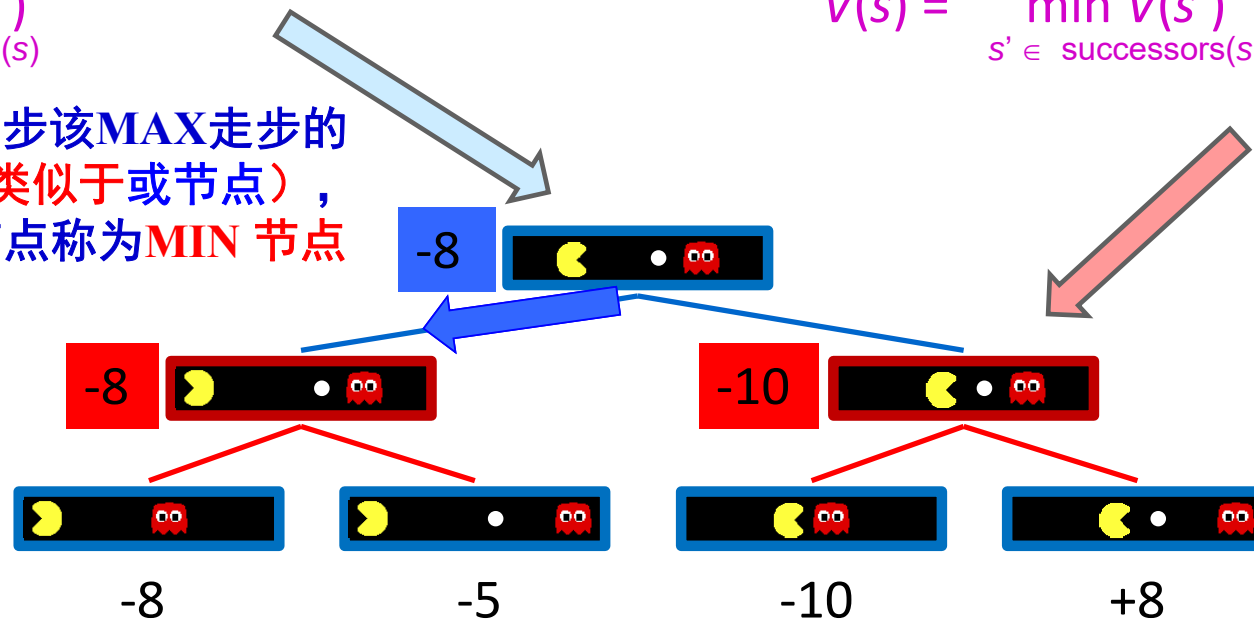
MAX nodes: under Agent's control

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

在博弈树中，那些下一步该MAX走步的节点称为**MAX 节点**（类似于或节点），下一步该MIN走步的节点称为**MIN 节点**（类似于与节点）。

MIN nodes: under Opponent's control

$$V(s) = \min_{s' \in \text{successors}(s)} V(s')$$



终止状态:

$$V(s) = \text{known}$$

极小化极大过程

对简单的博弈问题，可生成整个博弈树，找到必胜的策略。

对于复杂的博弈问题，不可能生成整个搜索树，如国际象棋，大约有 10^{120} 个节点。一种可行的方法是用当前正在考察的节点生成一棵部分博弈树，并利用估价函数 $f(n)$ 对叶节点进行估值。

极小化极大过程 (minimax value)

对叶节点的估值方法是：那些对MAX有利的节点，其估价函数取正值；那些对MIN有利的节点，其估价函数取负值；那些使双方均等的节点，其估价函数取接近于0的值。

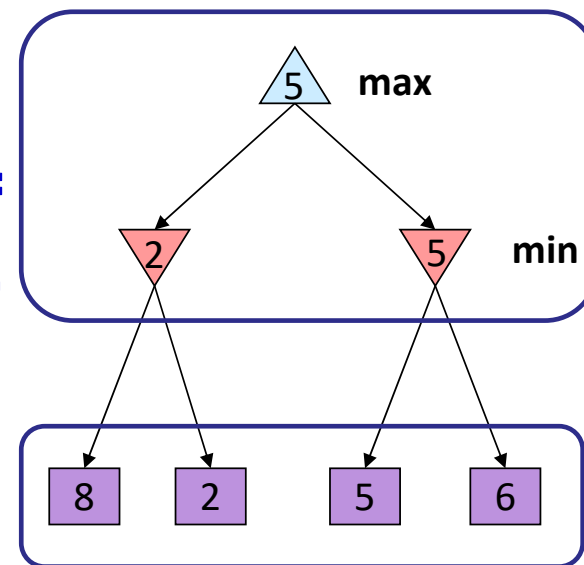
对非叶节点的值，必须从叶节点开始向上倒推。其倒推方法是：

对于MAX节点，由于MAX方总是选择估值最大的走步，因此，MAX节点的倒推值应取其后继节点估值的最大值。

对于MIN节点，由于MIN方总是选择估值最小的走步，因此，MIN节点的倒推值应取其后继节点估值的最小值。

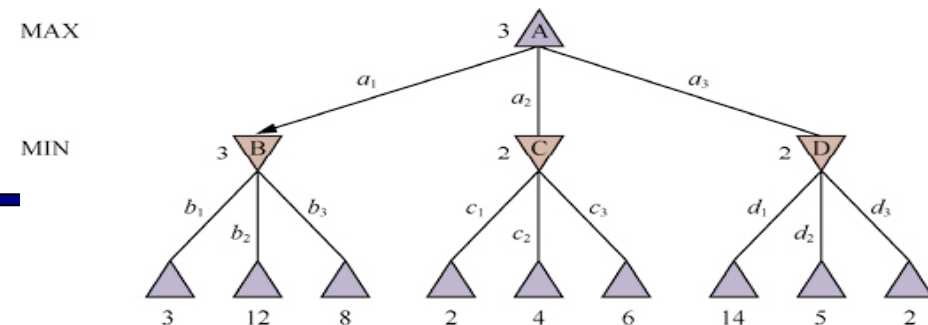
这样一步一步的计算倒推值，直至求出初始节点的倒推值为止。这一过程称为极小化极大值过程。

极小化极大值：
递归计算



叶节点：
部分博弈树

极小化极大算法



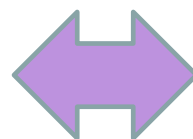
def max-value(state):

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{min-value}(\text{successor}))$

return v



def min-value(state):

initialize $v = +\infty$

for each successor of state:

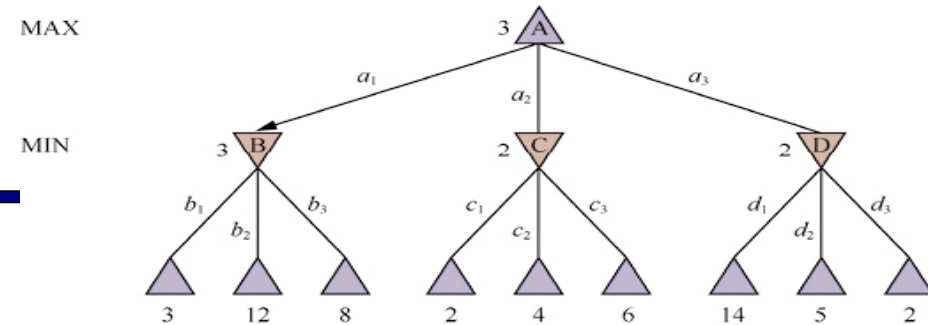
$v = \min(v, \text{max-value}(\text{successor}))$

return v

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

极小化极大算法(Dispatch)



def value(state):

if the state is a terminal state: return the state's utility

if the state is **MAX**: return **max-value(state)**

if the state is **MIN**: return **min-value(state)**

def max-value(state):

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return v

def min-value(state):

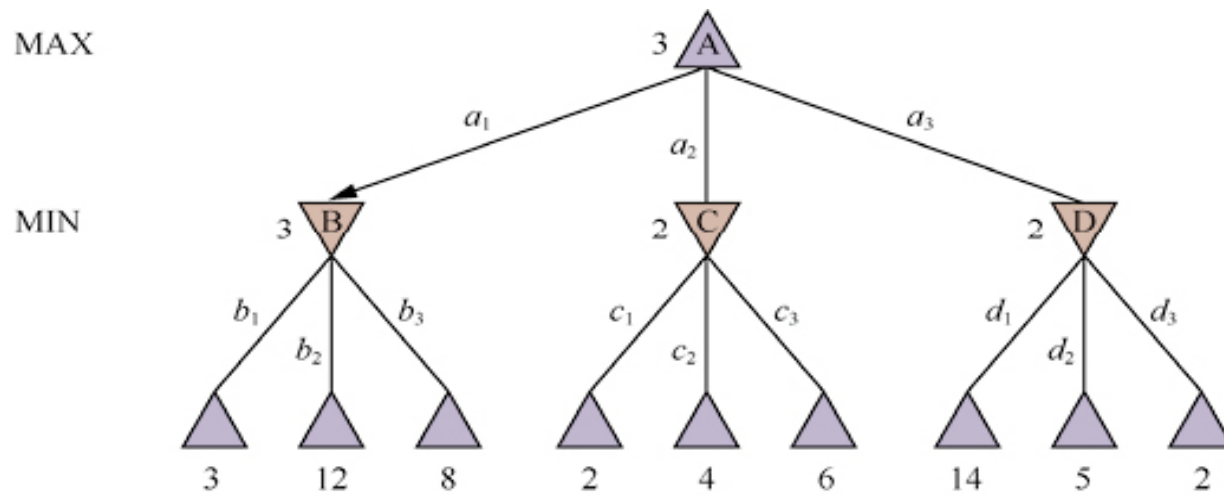
initialize $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}))$

return v

极小化极大算法

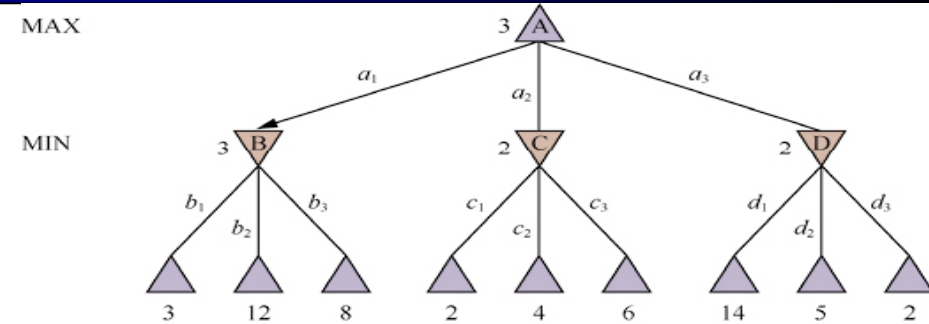
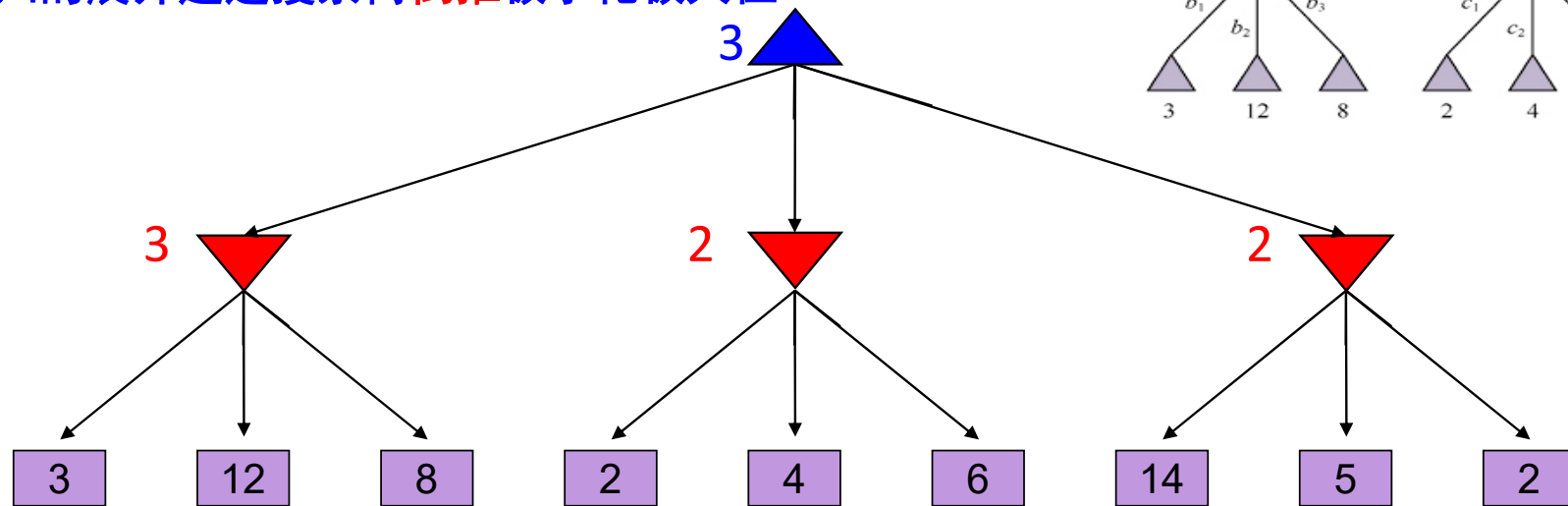


极小化极大 Example

递归算法：

一直向下进行到叶节点

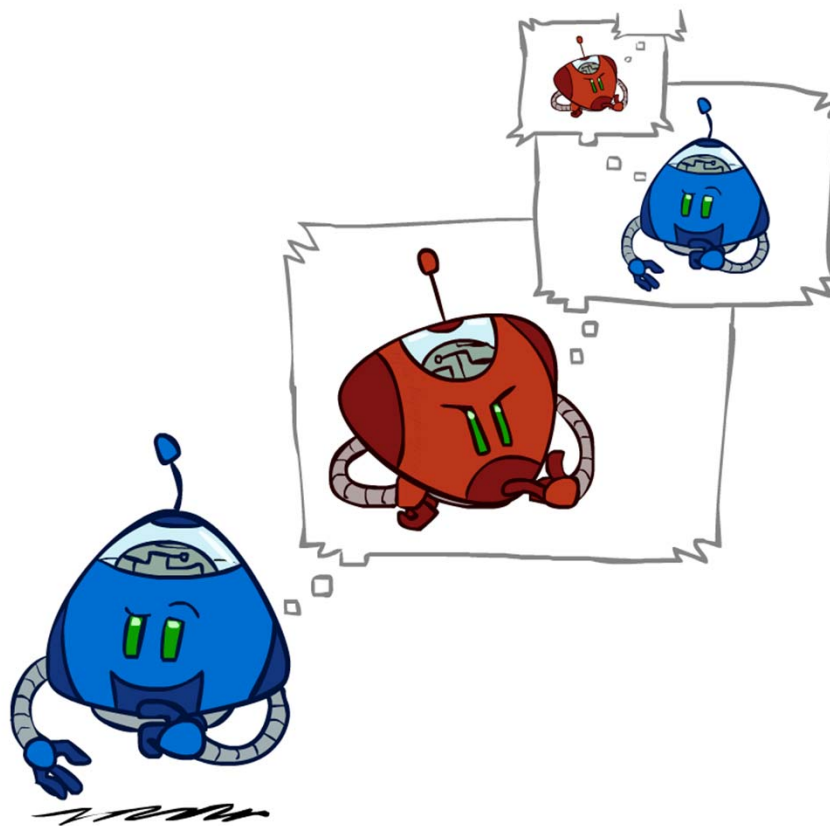
然后随着递归的展开通过搜索树倒推极小化极大值



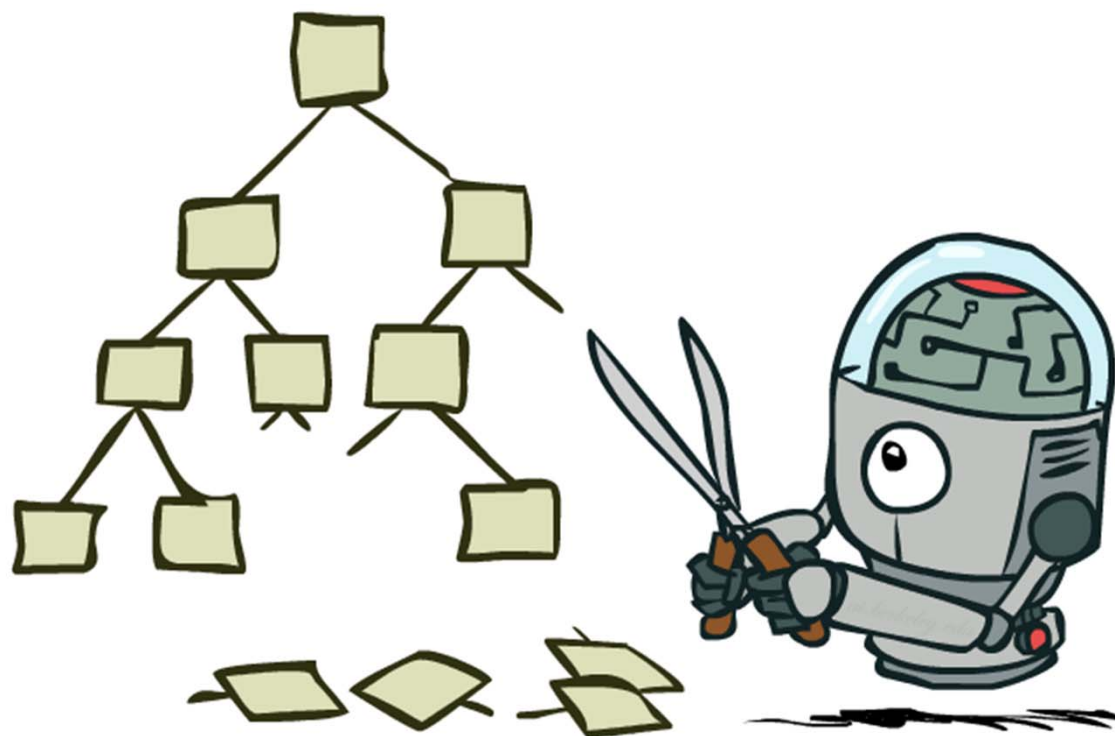
二层博弈树。▲ 节点为“max节点”，即轮到max移动，▼ 节点为“min节点”。终止节点显示max的效用值，其他节点标记有它们的极小化极大值。**max在根节点的最佳移动是 a_1** ，因为它指向极小化极大值最高的状态，而**min的最佳响应是 b_1** ，因为它指向极小化极大值最低的状态。

极小化极大算法效率

- 极小化极大算法效率?
 - 类似（穷举的）深度优先搜索
 - 时间复杂度: $O(b^m)$ 【b为最大分支因子】
 - 空间复杂度: $O(bm)$ 【m为树的深度】
- Example: 国际象棋, $b \approx 35$, $m \approx 80$
- $35^{80} \approx 10^{123}$
 - 精确解是完全不可行的
 - 人类也无法做到这一点
 - 那么我们如何下棋呢?



博弈树剪枝



$\alpha - \beta$ 剪枝

在极小极大搜索过程中，剪去一些没用的分枝，减少搜索空间，使得算法在同样时间内可以搜索更深层，这种技术被称为 $\alpha - \beta$ 剪枝。

α - β 剪枝的一般情况

(1) MAX节点（或节点）的 α 值为当前子节点的最大倒推值；

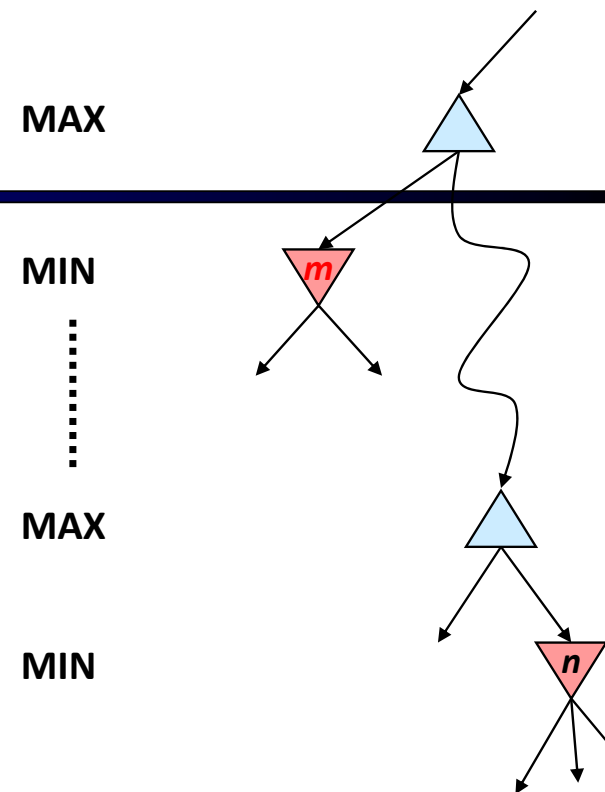
α =到目前为止路径上发现的MAX的最佳（极大值）选择

α ="至少" 【初始值为 $-\infty$ 】

(2) MIN节点（与节点）的 β 值为当前子节点的最小倒推值；

β =到目前为止路径上发现的MIN的最佳（极小值）选择

β ="至多" 【初始值为 $+\infty$ 】



α - β 剪枝的一般情况

(3) α - β 剪枝的规则如下：

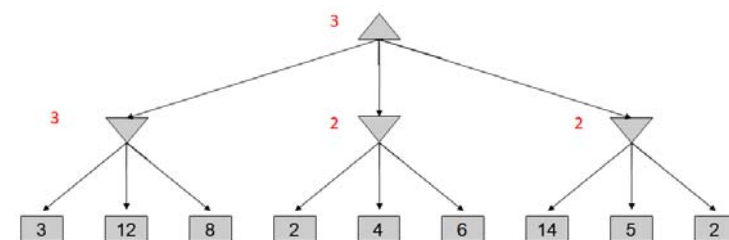
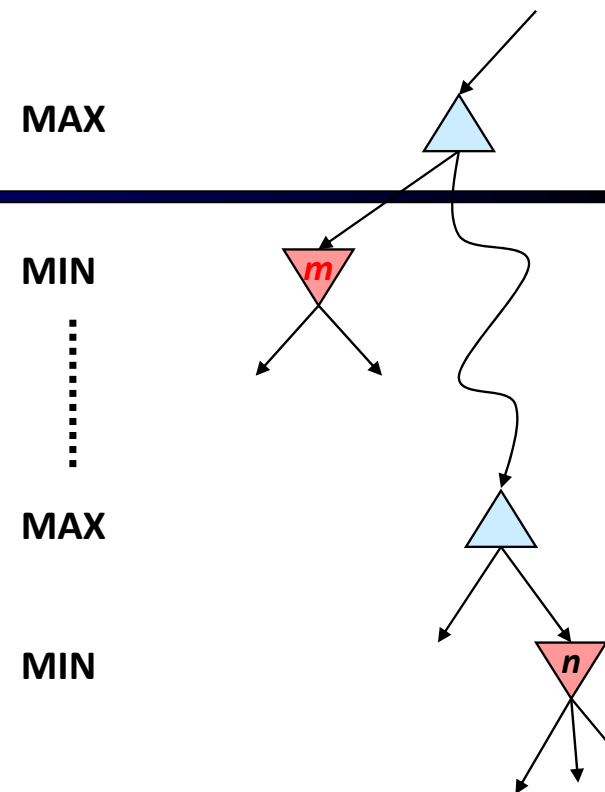
α 剪枝

任何**MAX**节点**n**的 α 值大于或等于它先辈节点的 β 值，
则**n** 以下的分枝可停止搜索，并令节点**n**的倒推值为 α 。

β 剪枝

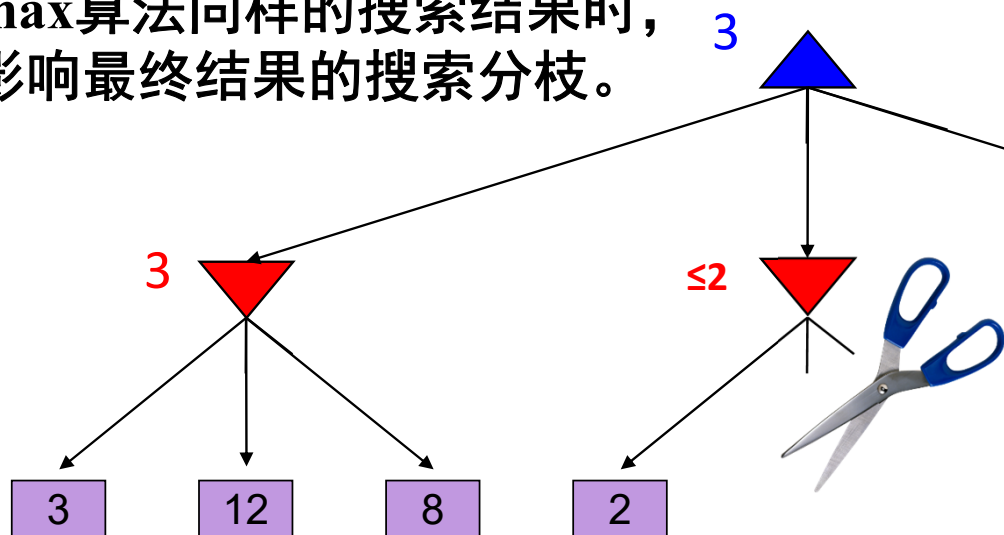
任何**MIN**节点**n**的 β 值小于或等于它先辈节点的 α 值，
则**n** 以下的分枝可停止搜索，并令节点**n**的倒推值为 β 。

α - β 搜索不断更新 α 和 β 的值，一旦当前节点的值比此时的 α （对于MAX）或 β （对于MIN）值更差，就剪掉该节点的剩余分支（即终止递归调用）



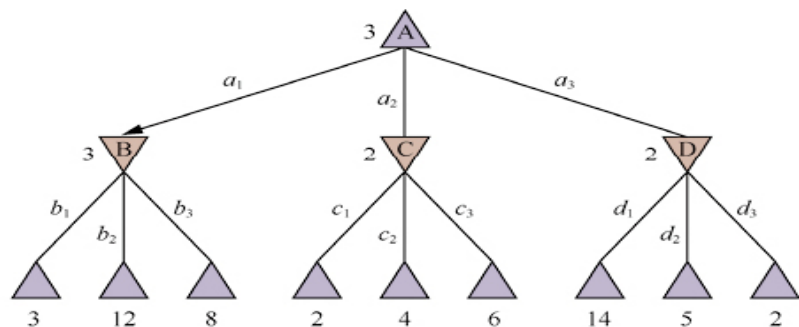
α - β 剪枝

α - β 剪枝搜索算法在Minimax算法中可**减少被搜索的节点数**，即在保证得到与原Minimax算法同样的搜索结果时，剪去了不影响最终结果的搜索分枝。



$$\begin{aligned}
 & \text{minimax}(\text{root}) \\
 &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\
 &= \max(3, \min(2, x, y), 2) \quad \min(2, x, y) \leq 2 \\
 &= 3 \quad \text{决策与叶节点x和y的值无关}
 \end{aligned}$$

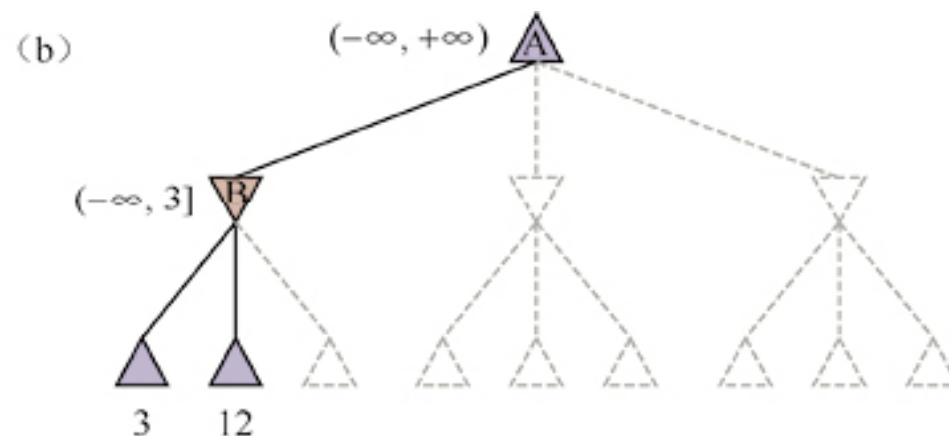
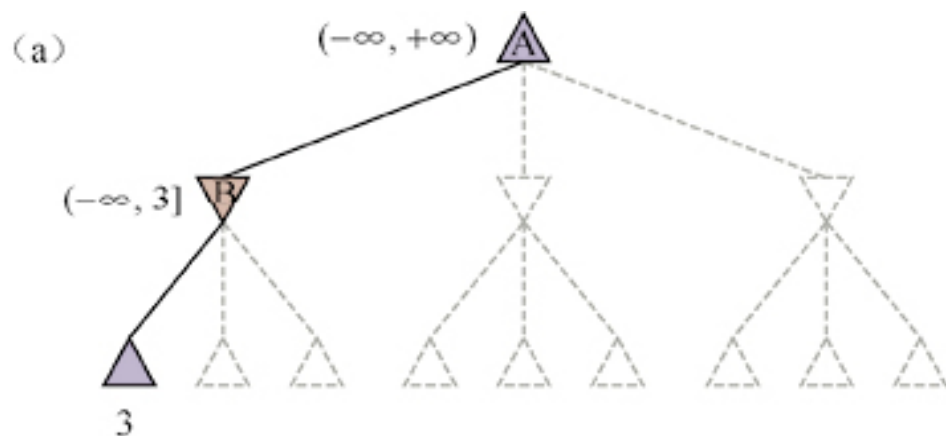
MAX
MIN



生成的顺序很重要：
如果先产生**好的动作**，
可以进行更多的剪枝。

博弈树的最优决策计算过程

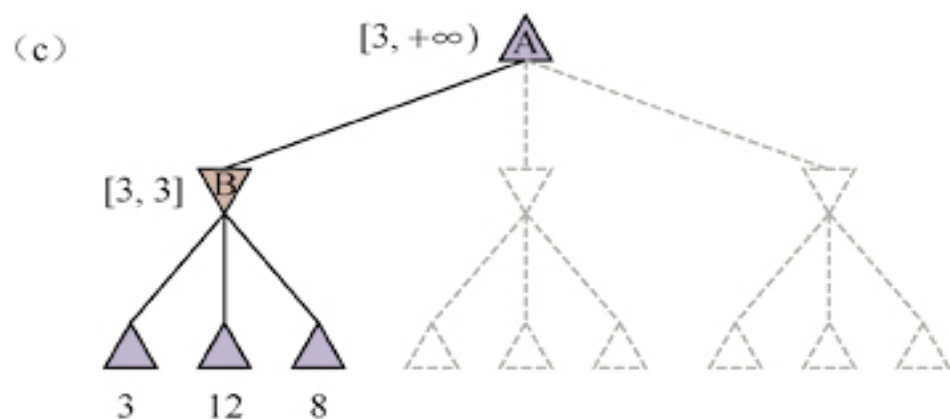
每一步都标有每个节点可能的值的范围



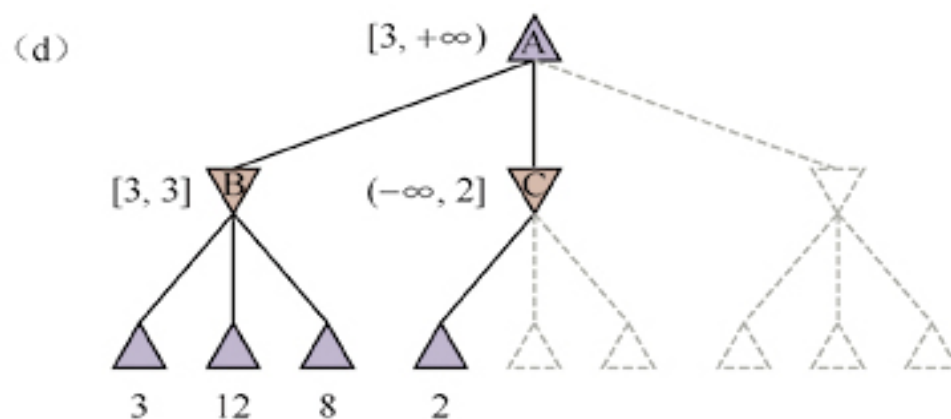
(a) B 下面的第一个叶节点值为3。
因此，作为min节点， B 的值最多为3。

(b) B 下面的第二个叶节点值为12，min将避免移动到该节点，
所以 B 的值仍然最多为3。

博弈树的最优决策计算过程

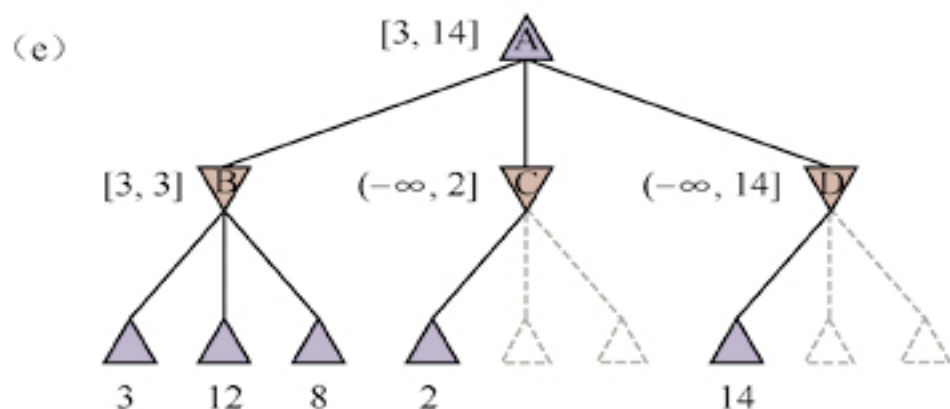


c) B下面的第三个叶节点值为8，此时我们已经检查完了B的所有后继状态，所以B的值就是3。
现在我们可以推断根节点的值至少是3，因为max在根节点处有值为3的选择。

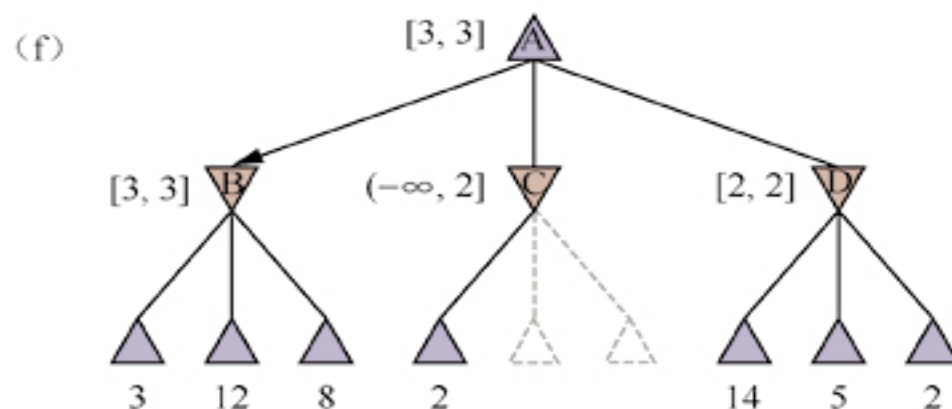


d) C下面的第一个叶节点值为2。因此，作为min节点，C的值最多为2。但是我们知道B的值为3，所以max永远不会选择C。
因此，没有必要再去检查C的其他后继状态。这是剪枝的一个实例。

博弈树的最优决策计算过程



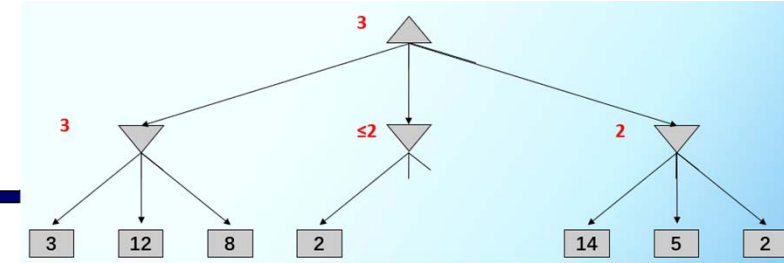
e) D下面的第一个叶节点值为14，所以D的值最多为14。这仍然高于max的最佳选择（即3），所以我们需要继续探索D的后继状态。注意，此时根节点的所有后继都有界，所以根节点的值也最多为14。



(f) D的第二个后继值为5，所以我们需要继续探索。第三个后继值为2，所以D的值就是2。

最终，max在根节点处的决策是移动到值为3的节点B。

α - β 搜索剪枝算法



α : MAX's best option on path to root
 β : MIN's best option on path to root

def max-value(state, α , β):

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$

if $v \geq \beta$ return v

$\alpha = \max(\alpha, v)$

return v

def min-value(state, α , β):

initialize $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

if $v \leq \alpha$ return v

$\beta = \min(\beta, v)$

return v



```

graph TD
    Root[ ] --- L1_1(( ))
    Root --- L1_2(( ))
    L1_1 --- L2_1[ ]
    L1_1 --- L2_2[ ]
    L1_2 --- L2_3[ ]
    L1_2 --- L2_4[ ]
    L2_1 --- L3_1(( ))
    L2_1 --- L3_2(( ))
    L2_2 --- L3_3(( ))
    L2_2 --- L3_4(( ))
    L2_3 --- L3_5(( ))
    L2_3 --- L3_6(( ))
    L2_4 --- L3_7(( ))
    L3_1 --- L4_1[3]
    L3_1 --- L4_2[17]
    L3_2 --- L4_3[2]
    L3_2 --- L4_4[12]
    L3_3 --- L4_5[15]
    L3_4 --- L4_6[25]
    L3_4 --- L4_7[0]
    L3_5 --- L4_8[2]
    L3_6 --- L4_9[5]
    L3_7 --- L4_10[3]
    L3_7 --- L4_11[2]
    L3_7 --- L4_12[14]
  
```

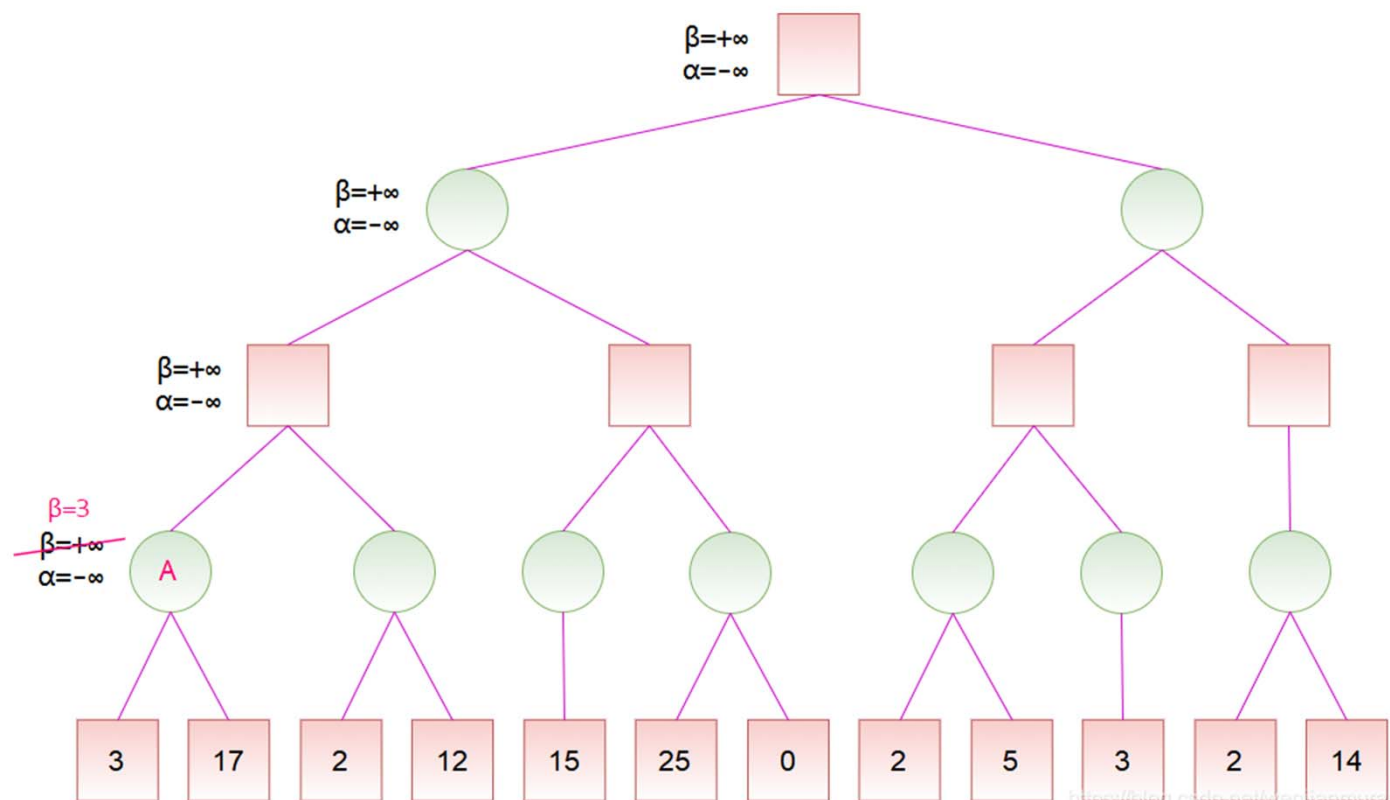
α - β 剪枝实例



MAX



MIN



def min-value(state , α , β):

initialize $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

if $v \leq \alpha$ return v

$\beta = \min(\beta, v)$

return v

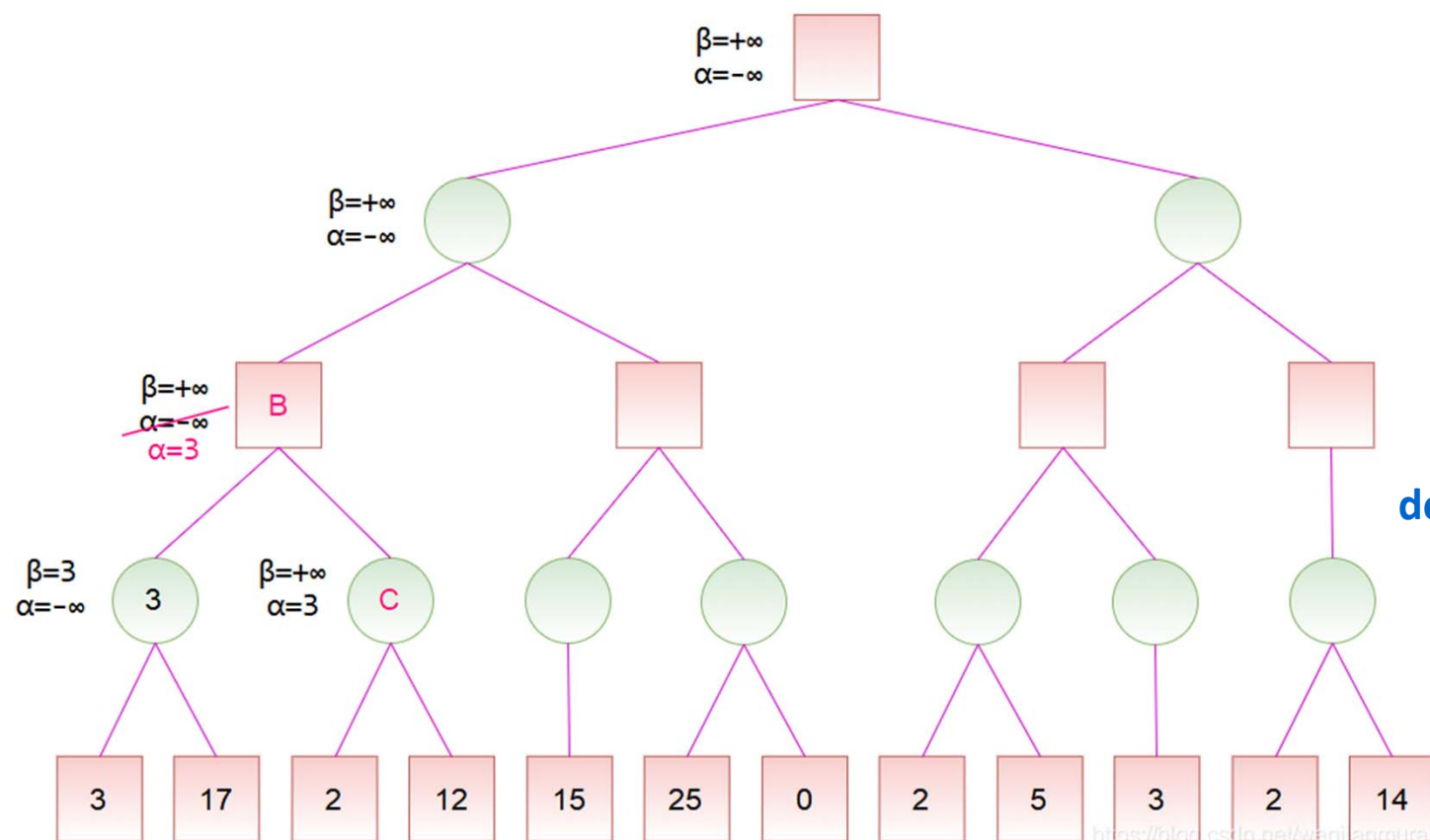
α - β 剪枝实例



MAX



MIN



def min-value(state, α , β):

initialize $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

if $v \leq \alpha$ return v

$\beta = \min(\beta, v)$

return v

def max-value(state, α , β):

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$

if $v \geq \beta$ return v

$\alpha = \max(\alpha, v)$

return v

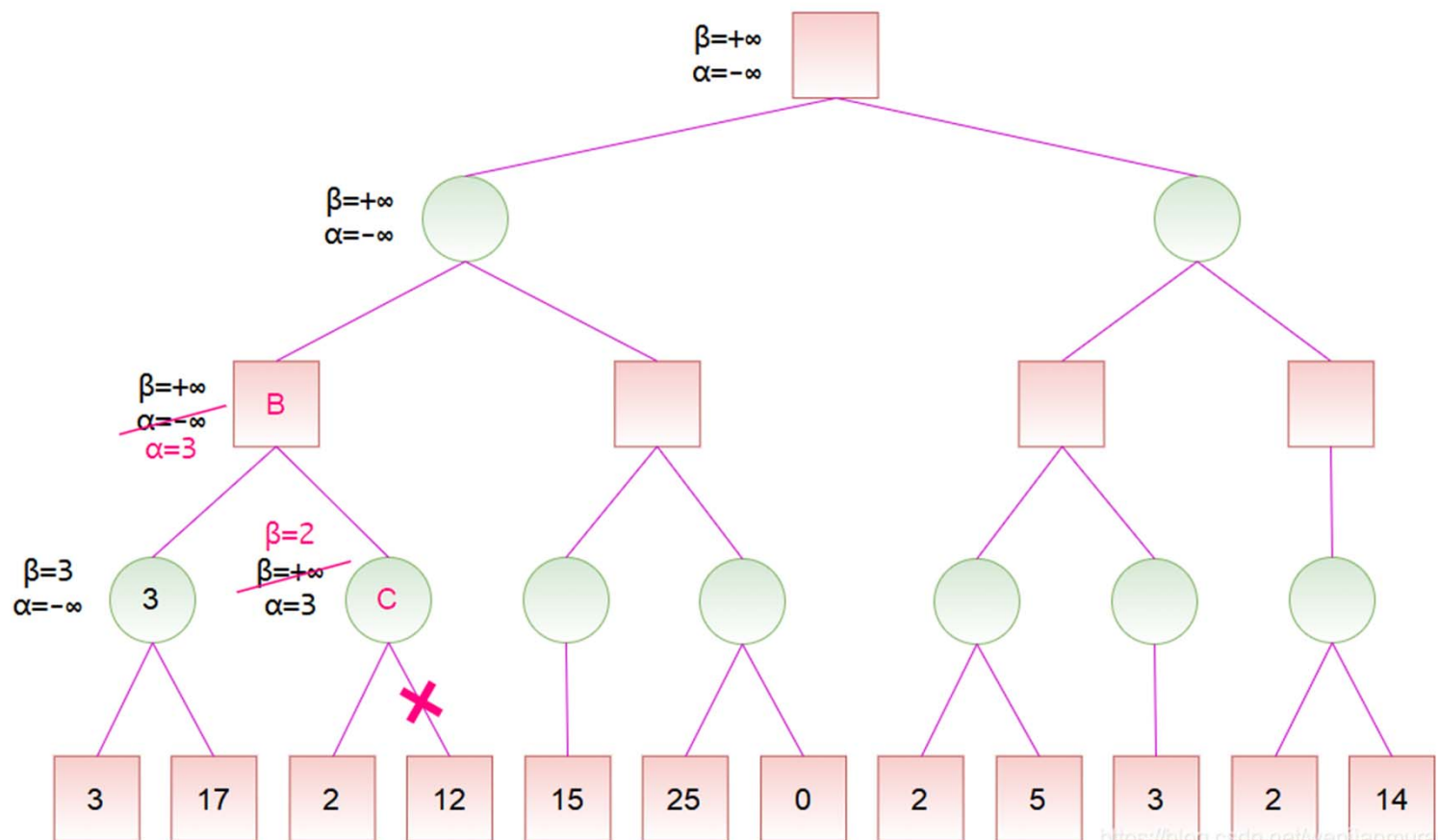
α - β 剪枝实例



MAX



MIN



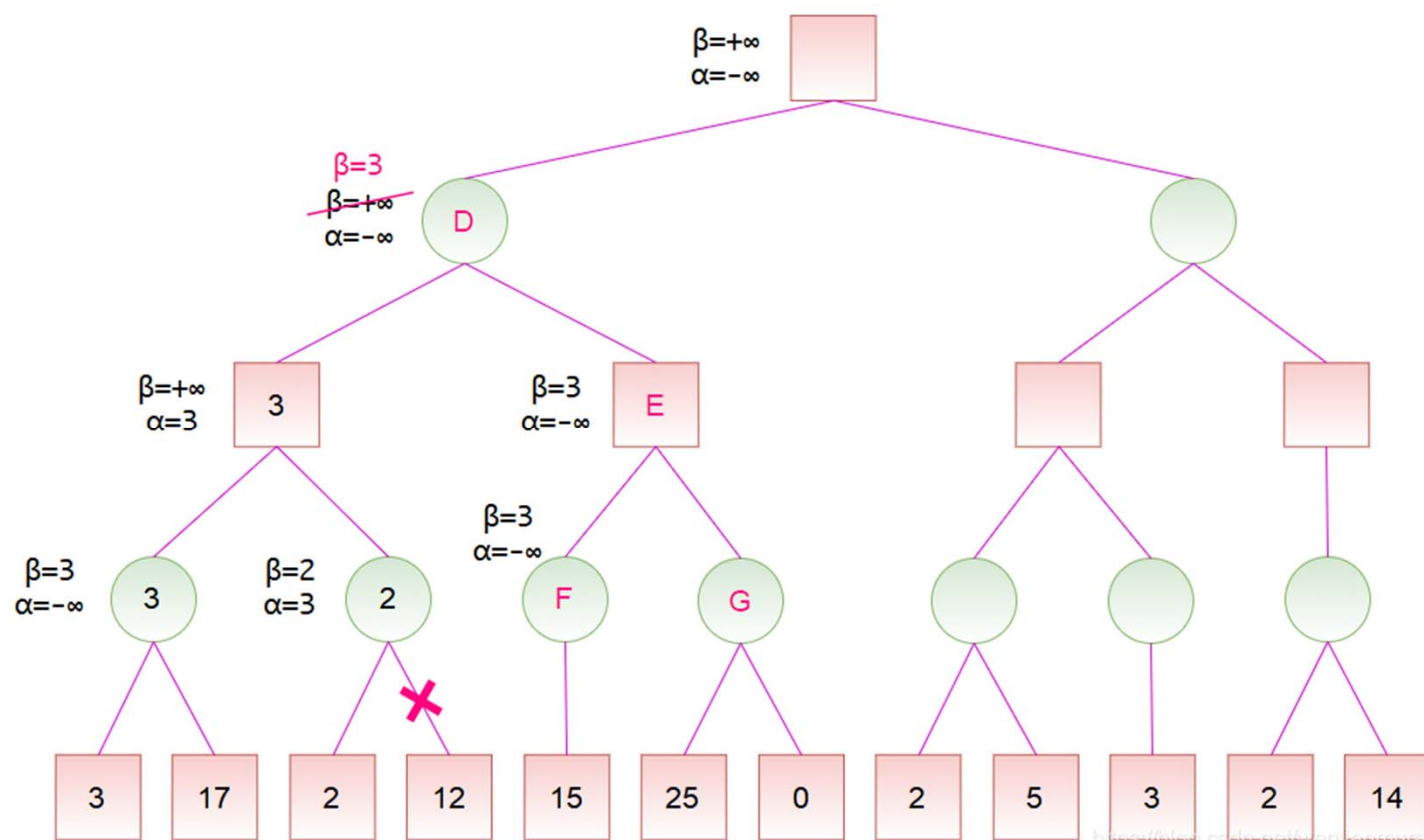
α - β 剪枝实例



MAX



MIN



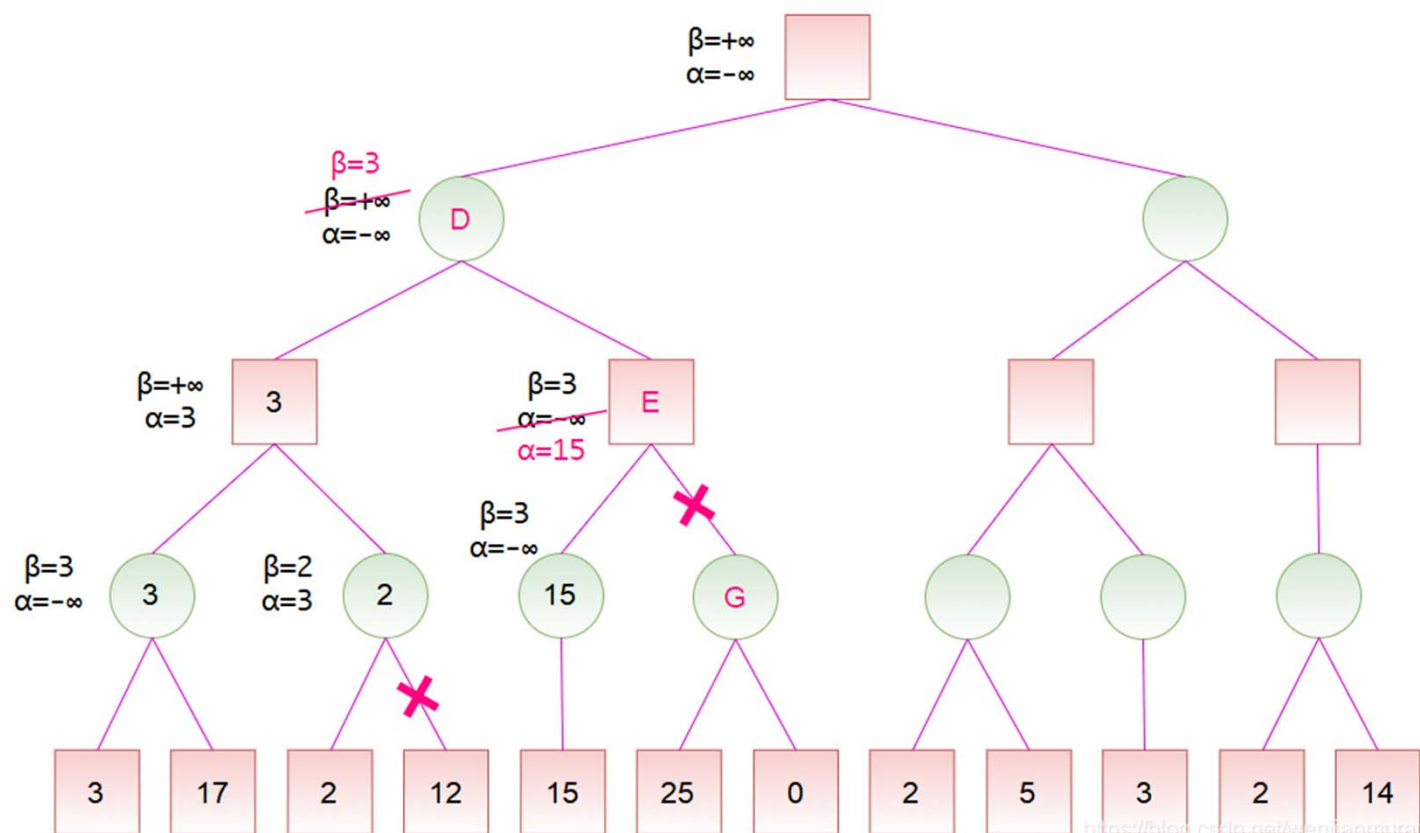
α - β 剪枝实例



MAX



MIN



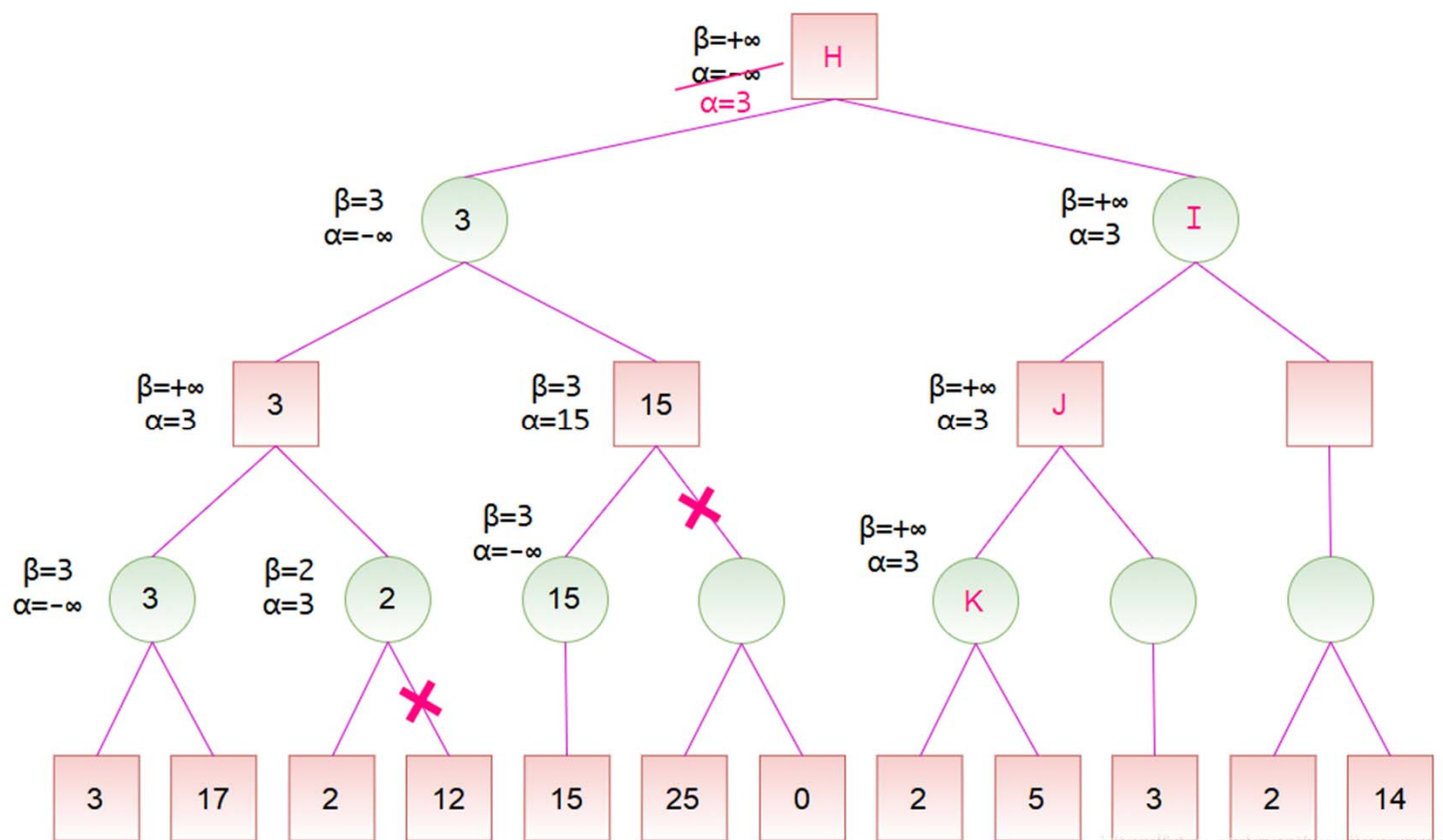
α - β 剪枝实例



MAX



MIN



<https://blog.csdn.net/wenjianmuran>

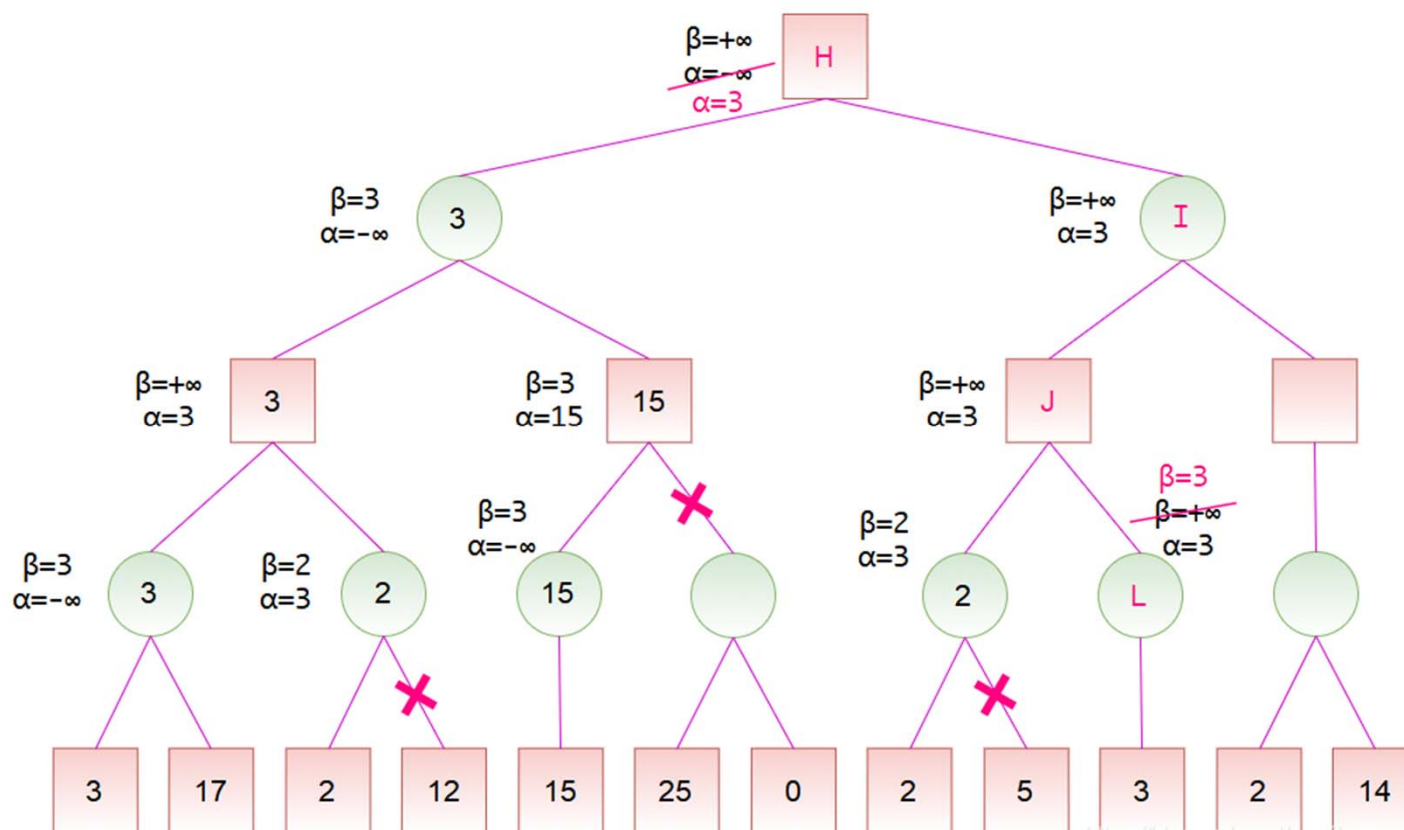
α - β 剪枝实例



MAX



MIN



<https://blog.csdn.net/wenjianmuran>

11



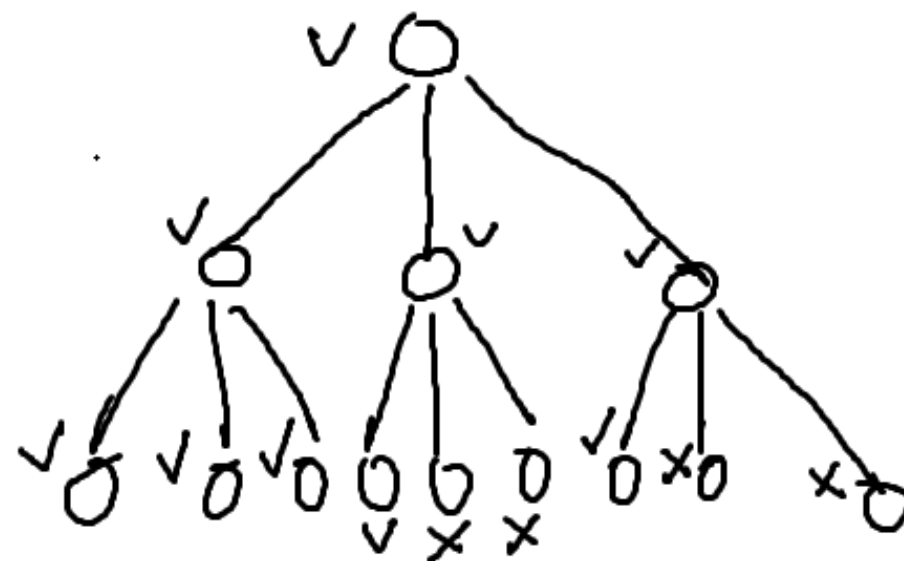
The diagram shows a minimax search tree with the following structure and values:

- Root (Max node H):** $\beta = +\infty$, $\alpha = -\infty$, $\alpha = 3$.
 - Left child (Min node 3): $\beta = 3$, $\alpha = -\infty$.
 - Left child (Max node 3): $\beta = +\infty$, $\alpha = 3$.
 - Left child (Min node 3): $\beta = 3$, $\alpha = -\infty$.
 - Left child (Max node 3): value 3.
 - Right child (Max node 17): value 17.
 - Right child (Min node 2): $\beta = 2$, $\alpha = 3$.
 - Left child (Max node 2): value 2.
 - Right child (Max node 12): value 12. (Pruned, marked with a red X)
 - Right child (Max node 15): $\beta = 3$, $\alpha = 15$.
 - Left child (Min node 15): $\beta = 3$, $\alpha = -\infty$.
 - Left child (Max node 15): value 15.
 - Right child (Max node 25): value 25. (Pruned, marked with a red X)
 - Right child (Min node 0): value 0. (Pruned, marked with a red X)
 - Right child (Min node I): $\beta = 3$, $\beta = +\infty$, $\alpha = 3$.
 - Left child (Max node 3): $\beta = +\infty$, $\alpha = 3$.
 - Left child (Min node 2): $\beta = 2$, $\alpha = 3$.
 - Left child (Max node 2): value 2.
 - Right child (Max node 5): value 5. (Pruned, marked with a red X)
 - Right child (Min node 3): $\beta = 3$, $\alpha = 3$.
 - Left child (Max node 3): value 3.
 - Right child (Max node 14): value 14. (Pruned, marked with a red X)
 - Right child (Min node): empty node.
 - Left child (Max node 2): value 2.
 - Right child (Max node 14): value 14.

(原本Minimax搜索树中有26个节点，经过 α - β 剪枝后只扩展（访问）了其中17个节点，
可见 α - β 剪枝技术能够有效地减少搜索树中节点的数目)

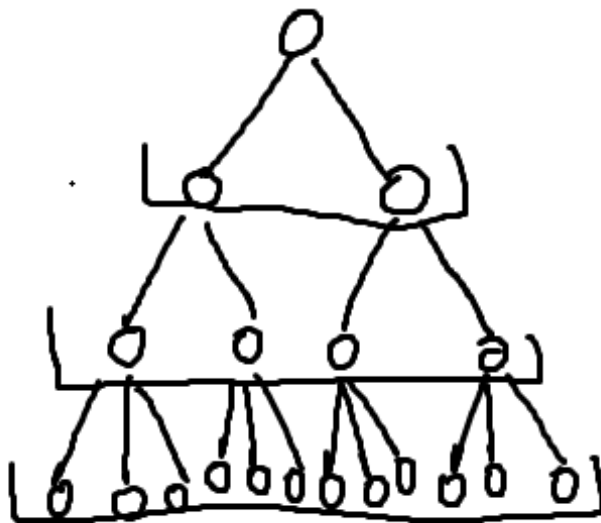
α - β 剪枝算法的性质

- 高度依赖于移动的顺序
- 最坏的情况：没有剪枝 $O(b^m)$
- 最好的情况：始终首先检查最佳的移动
 - 仍然需要检查第一名玩家的每一个动作
 - 只需要检查第二名玩家的一个动作
 - $O(b \cdot 1 \cdot b \cdot 1 \dots) = O(b^{\frac{m}{2}})$



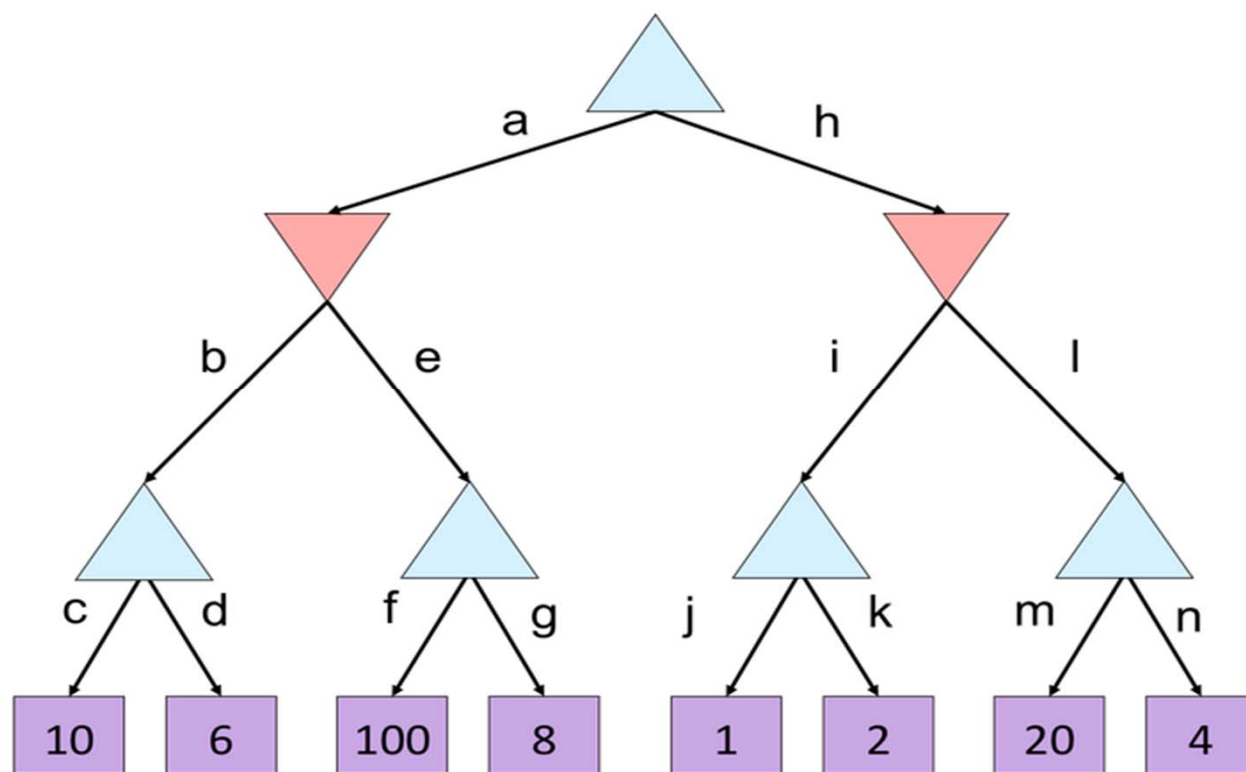
α - β 剪枝算法的性质

- 平均情况: $O(b^{\frac{3m}{4}})$
- 非常简单的排序通常就可以实现 $O(b^{\frac{m}{2}})$
 - 采取迭代加深算法
 - 有效性: 将分支因子减少到 $\sqrt{b}$; 或使搜索深度加深两倍。



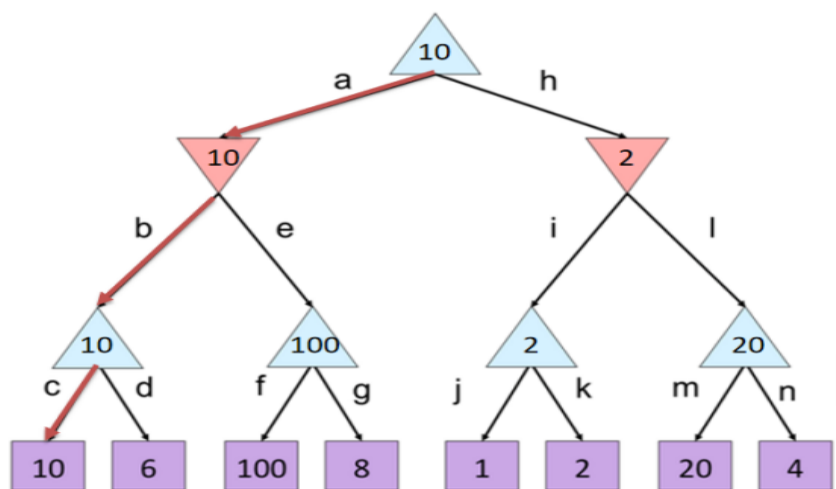
对每层节点重新排序

课堂练习 (请分别用极小极大算法和 α - β 剪枝算法计算博弈树)

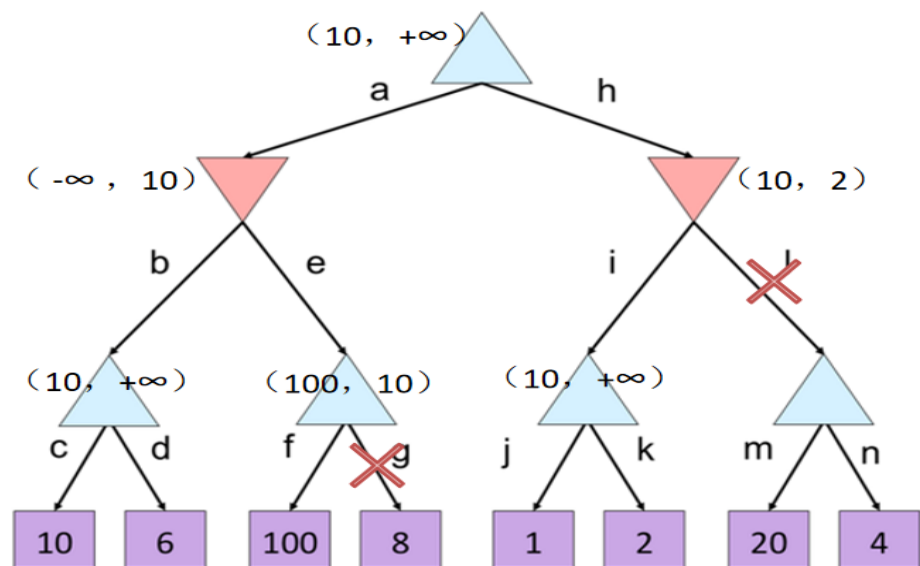


参考答案

极小极大算法



α - β 剪枝算法



谢谢！